



TUM School of Computation, Information and Technology
Professorship of Data Engineering

Bachelor Thesis

**Thesis for the attainment of the academic degree
Bachelor of Science (B.Sc.) Information Engineering**

Root-Cause Localization for SHACL Referencing-Constraint Failures

Author: Mariya Kezdekbayeva
Supervisor: Jin Ke, M.Sc.
Examiner: Prof. Maribel Acosta
Submission Date: 24.09.2026



Declaration of Authorship

I hereby declare that I have written the present thesis with the title “Root-Cause Localization for SHACL Referencing-Constraint Failures” independently and without the use of other than the indicated aids. I affirm that I have not used any sources other than those indicated and that all passages taken verbatim or in spirit from published and unpublished writings are identified as such. Furthermore, I assure that the work has not yet been submitted in the same or similar form in the context of another examination. I am aware that in the event of deception, the thesis will be graded as “failed”.

24.09.2026, Heilbronn

date, place



signature

Statement on the Use of AI-Based Tools

The following AI-based tools were used in the preparation of this thesis. All content generated or suggested by these tools was critically reviewed, verified against the primary sources, and revised by the author, who bears full responsibility for the final text.

Claude Code (Anthropic) and Codex (OpenAI). Used to assist with refining and checking the code for this thesis.

ChatGPT (OpenAI) and Claude (Anthropic). Used to refine style and to check the grammar of the author's drafts.

Abstract

This thesis investigates root-cause localization for SHACL validation failures involving referencing constraints. It develops a deterministic explanation-tree representation that traces reported `sh:node` failures to concrete SHACL Core violations, including the relevant path, value node, reference chain and local repair hint where supported. The approach handles nested references, missing `sh:detail` information in external reports, cyclic references and diamond-shaped reference structures, and presents the reconstructed failures through textual, tabular, JSON, summary and interactive HTML outputs. The evaluation combines controlled fixtures with LUBM-derived graphs to assess correctness, scalability and explanation overhead.

Contents

Abstract	iv
1. Introduction	1
2. Preliminaries	5
2.1. RDF Graphs	5
2.2. The SHACL Validation Model	5
2.3. Constraint Components and Leaf Failures	7
2.4. The sh:node Referencing Constraint	8
2.5. Terminology Used in This Thesis	9
2.6. External Validation Reports and Optional sh:detail	11
2.7. Recursive and Cyclic Shape References	11
3. Related Work	12
3.1. Repairs, Provenance and Proofs	12
3.2. Generated Explanations and Report Exploration	13
3.3. Dependent Shapes, Recursion and Scale	14
3.4. Comparison and Research Gap	15
4. Our Approach	17
4.1. Pipeline Overview	17
4.2. Extracting Top-Level Results	19
4.3. The Explanation-Tree Data Model	19
4.4. Recursive Expansion	20
4.5. Node-Level Versus Property-Level References	23
4.6. External Reports and the Fallback Path	25
4.7. Deduplicating Diamond-Shaped References	33
4.8. Rendering and Output Modes	36
4.9. The Interactive HTML Report	40
4.10. Summary	43

5. Experiments	44
5.1. Experimental Setup	44
5.1.1. Functional Evaluation	44
5.1.2. Performance Evaluation	44
5.1.3. Implementation Details	45
5.1.4. Metrics	45
5.1.5. Protocol and Reproducibility	46
5.2. Dataset and Shape Characteristics	46
5.2.1. Schema Defects and the Corrected Variants	48
5.3. Results and Discussion	49
5.3.1. RQ1: Functional Coverage of Referencing Patterns	49
5.3.2. RQ2: Reconstruction from External Reports	52
5.3.3. RQ3: Scaling with Data Size and Schema Reach	58
5.3.4. RQ4: Cost of the Explanation Layer	61
5.3.5. RQ5: Consistency of Findings Across Scale	63
5.4. Threats to Validity and Summary	66
5.4.1. What the reported numbers mean	66
5.4.2. Whether the measured effects are real	68
5.4.3. How far the results generalize	69
5.4.4. What the explanation guarantees	70
5.4.5. Reproducibility	72
5.4.6. Summary of findings	72
6. Conclusions and Future Work	73
6.1. Contributions	73
6.2. What the evaluation shows	73
6.3. Future work	74
6.3.1. Strengthen correctness, then broaden coverage	74
6.3.2. Test reconstruction across validators	75
6.3.3. Make large explanations usable	75
Appendix	76
A. Implementation Map	76
B. Worked Examples and Reconstruction Cases	77
B.1. Reference display rules and text examples	77
B.2. TC7 expansion trace	78
B.3. Incomplete reports: what survives	79
B.4. TC7 JSON example	81

B.5. Evaluation probes: ordering and shared paths	82
C. CSV Export Columns	83
D. Supplementary Evaluation Tables	84
E. Per-Fixture Assertion Coverage	87
F. Corrections to the Supplied LUBM Schemas	89
List of Figures	90
List of Tables	91
List of Procedures	93
Bibliography	94

1. Introduction

Motivation. The Shapes Constraint Language (SHACL) [KK17] validates RDF graphs against shapes and reports which nodes violate which constraint components. For direct constraints, a validation result often already identifies the focus node, property path and offending value. Referencing constraints complicate this picture. Through `sh:node`, a shape can require a focus node, or a value reached through a property, to conform to another shape. When that referenced shape fails, the outer result states that conformance failed but does not itself identify the concrete constraint inside the referenced shape that caused the failure. De Meester et al. state the broader problem directly: where a nested shape is violated, the validation report records only the top-level failure [De +21].

The missing cause may be recoverable from nested `sh:detail` results, but this information is not guaranteed. SHACL leaves `sh:detail` optional [KK17; Lab+18], and validators such as Apache Jena [Apa26] can report a failed referenced shape without embedding the nested results that explain why. `sh:node` can be used in two ways. At node level, the same focus node is checked against another shape. At property level, a value reached through a property becomes the node checked against the referenced shape. A useful explanation must therefore preserve not only the referenced shape but also the path and, where applicable, the value node to which the nested failure belongs.

Referencing may also be multi-level, cyclic or diamond-shaped. A referenced shape can itself contain another `sh:node`, pushing the concrete violation several levels below the top-level result. Conversely, two branches can reach the same referenced shape and cause the same underlying failure to appear more than once. These cases turn a valid SHACL report into a localization problem. The report shows that a referenced shape failed, but the user still has to determine the concrete failure, the affected node and path, and the reference chain through which the failure was reached.

This matters particularly for large reports, where manual traversal is impractical. Chapter 5 evaluates the approach on LUBM-derived graphs [GPH05] containing more than one million and four million triples. These experiments show that SHACL validation can produce very large reports. In the largest case, pySHACL produces 2,281,457 results. The explanation pipeline reorganizes them into 87,396 distinct leaf failures, reducing the number of presented failures by roughly a factor of 26. In such settings, simply exposing all validation results does not solve the explanation problem:

the relevant information may still be buried beneath intermediate `sh:node` results or duplicated along different routes. The purpose of this thesis is therefore not to replace SHACL validation, but to make failures involving referencing constraints easier to trace from the reported `sh:node` violation to the concrete constraint that caused it.

Problem statement. The input consists of a data graph, a shapes graph containing `sh:node` constraints, and a SHACL validation report. The report may come from pySHACL [SC26] or from an external validator. The goal is to reconstruct the concrete SHACL Core violation behind each reported referencing failure. The reconstruction should identify the failed constraint component, the relevant `sh:path`, the value node where available and the chain of referenced shapes connecting the original report result to the failing constraint.

Three complications are central. First, nested results may be absent because `sh:detail` is optional, so the missing information must be recovered from the data and shapes graphs rather than merely parsed from the report. Second, several reference chains may reach the same concrete violation; such diamond-shaped cases should not produce misleading duplicates, but the alternate routes must still be preserved because they describe different dependencies in the shapes graph. Third, referencing shapes may be cyclic, so reconstruction must terminate without discarding concrete failures that remain reachable within a valid branch. The problem is therefore both structural and practical: an explanation must reconstruct missing nesting, retain meaningful reference context and avoid presenting report-level redundancy as distinct root causes.

In this thesis, the *root cause* of a referencing failure is the concrete leaf-level SHACL Core constraint violation that explains the reported failure, together with the chain of shapes and property paths through which it was reached. For a direct SHACL Core violation, the root cause is the violation itself. Here, “root cause” refers only to localization of the concrete SHACL violation. The thesis does not claim to compute a minimal repair or identify one uniquely responsible data edit.

Scope. The work focuses on SHACL Core constraints and `sh:node` referencing constraints. It covers nested reference chains, property-level `sh:node`, multiple referenced-shape failures on the same focus node, diamond-shaped references, reports with or without `sh:detail` and external reports produced by Apache Jena and TopBraid. SHACL-SPARQL constraints, full support for `sh:and`, `sh:or`, `sh:not` and `sh:xone`, full support for `sh:qualifiedValueShape` and property paths beyond what the current report and reconstruction logic can handle are out of scope. Chapters 2 and 5 define these boundaries in more detail.

Research questions. The thesis evaluates the approach through five questions:

- **RQ1.** Across the referencing patterns the tool claims to support, does the explanation tree correctly recover the failed constraint, the `sh:path`, the value node, the

chain of referenced shapes and the repair hint?

- **RQ2.** Can the tool reconstruct the same explanation tree from an external, non-pySHACL validation report that lacks `sh:detail`?
- **RQ3.** How does end-to-end runtime scale with data size, schema reach and the validation-result volume that combination produces on realistic large graphs?
- **RQ4.** Does building and rendering the explanation tree add meaningful overhead on top of the SHACL validation it post-processes?
- **RQ5.** Does the dominant failure and repair pattern stay the same as the data scales up?

Here, schema reach is the extent to which a shapes graph applies to populous parts of the data graph. The first two questions concern functional correctness on controlled fixtures. The last three concern performance and observed failure patterns on larger LUBM-derived graphs.

Approach overview. The central idea is to treat the validation report as input rather than as the final explanation. The proposed system rebuilds the report into an *explanation tree*. Interior nodes represent traversed `sh:node` references, while leaves represent concrete SHACL Core violations. Each leaf records the failed constraint, path, relevant value node and reference chain, and can carry a repair hint when one can be derived from the failed component.

Where the report already contains nested `sh:detail`, the system follows that structure until concrete constraint violations are reached. If nested details are missing, the system reconstructs them using the data graph and shapes graph. It re-validates the relevant node against the referenced shape. For property-level `sh:node`, this means attributing the recovered failure to the referenced value node rather than incorrectly leaving it on the outer focus node. Cyclic references are guarded during traversal, and duplicate leaves reached through diamond-shaped references are collapsed while their alternate chains are retained.

The same representation also accommodates direct SHACL Core violations, which appear as leaves without a reference chain. This keeps direct and referenced failures comparable while preserving the distinction between them. The explanation is derived structurally from the validation report, data graph, and shapes graph. The approach is therefore rule-based and does not use a generative explanation step. Given the same inputs, it produces the same failures and reference chains. That is a statement about content and not about serialization, and Section 5.4 makes the distinction precise. Repair hints are attached only where the failed component supports the tool's predefined

hint logic; they are aids for acting on localized failures, not proofs of globally minimal repairs.

The resulting tree can then be rendered or aggregated for inspection. The implementation additionally supports several presentation forms, but these are views over the same reconstructed structure rather than separate explanation methods. Chapter 4 explains those mechanics and output formats in detail.

Contributions. This thesis makes five contributions:

- a rule-based explanation-tree reconstruction method for localizing `sh:node` failures to concrete SHACL Core violations, derived from the validation report and shapes graph without a generative step;
- a fallback mechanism for tested external-report patterns in which `sh:detail` is absent, using targeted re-validation to recover omitted nested results;
- deduplication of failures reached through diamond-shaped references while preserving alternate reference chains;
- user-facing representations of the reconstructed explanation, including repair-oriented information and an interactive report for larger result sets;
- a functional evaluation over 53 fixtures in ten categories, using an automated suite of 52 tests. The tests verify constraint components, paths, values, repair phrasing, and stored explanations. The evaluation additionally measures performance at LUBM scale.

These contributions are intentionally narrower than general SHACL explanation or repair. The system does not redefine SHACL semantics, replace the validator, generate minimal repairs or claim coverage beyond the stated constraint scope.

Thesis structure. Chapter 2 introduces the RDF and SHACL concepts needed by the thesis, including `sh:node`, validation reports, the project vocabulary, missing `sh:detail` and cyclic references. Chapter 3 positions the work against research on SHACL explanations, repairs, provenance, generated natural-language explanations, alternative validation and formalization strategies such as Validatrr, validation performance, visualization and related shape languages. Chapter 4 presents the explanation-tree approach and its implementation. Chapter 5 evaluates functional correctness and performance against the five research questions. Chapter 6 summarizes the contributions, states what the evaluation shows and outlines future work.

2. Preliminaries

This chapter introduces the RDF and SHACL concepts needed to trace a reported `sh:n`ode failure to a concrete constraint violation.

2.1. RDF Graphs

An RDF graph is a set of subject–predicate–object triples [CWL14]. Subjects are IRIs or blank nodes, predicates are IRIs, and objects may also be literals. IRIs are often abbreviated using prefixes, for example `ex:alice`. Blank nodes commonly represent anonymous property shapes. Literals may also have datatypes such as `xsd:integer`. The **data graph** contains the facts to be checked, and the **shapes graph** contains their constraints. The examples use Turtle syntax [PC14].

2.2. The SHACL Validation Model

SHACL validates an RDF data graph against constraints in a shapes graph [KK17; Lab+18].

A **node shape** constrains a node directly and is commonly represented as an instance of `sh:NodeShape`. A **property shape** constrains values reached from a node through a path. Its `sh:path` identifies the path, which is most commonly a property IRI. A node shape can attach property shapes through `sh:property`. Property shapes are often blank nodes embedded in a node shape, although they may also be named resources. This distinction matters when reading validation reports. `sh:sourceShape` may refer either to a node shape or to the property shape on which the failed constraint was declared.

A **target declaration** determines which data nodes a shape applies to. For example, `sh:targetNode` selects a particular node and `sh:targetClass` selects instances of a class. A node currently being checked against a shape is the **focus node**. When a property shape is evaluated, the nodes reached from the focus node through its `sh:path` are **value nodes**. This distinction becomes important for property-level `sh:node`, where a value node is itself checked against another shape.

Validation produces a `sh:ValidationReport` whose `sh:conforms` value states the outcome. Its `sh:result` links point to the top-level `sh:ValidationResult` nodes. Nested results may be linked through `sh:detail` and describe causes below these top-level results [KK17; Lab+18]. Table 2.1 lists the report properties used in this thesis.

Table 2.1.: Validation-result properties used in this thesis, adapted from [Lab+18].

Property	Meaning
<code>sh:focusNode</code>	Node being validated when the result was produced.
<code>sh:resultPath</code>	Path associated with the violation, where applicable.
<code>sh:value</code>	Value node associated with the violation, where applicable.
<code>sh:sourceShape</code>	Shape whose constraint produced the result.
<code>sh:sourceConstraintComponent</code>	Constraint component that was violated.
<code>sh:resultMessage</code>	Optional human-readable message supplied by the shape.
<code>sh:resultSeverity</code>	Severity of the result.
<code>sh:detail</code>	Optional link to nested validation results giving further detail.

Node-level results may omit `sh:resultPath` and `sh:value`; in property-level results, these identify the path and relevant value. Results also carry a severity: `sh:Violation` (the default), `sh:Warning` or `sh:Info` [Lab+18]. Severity does not affect how the explanation is expanded. A direct constraint and its abbreviated result illustrate the report model:

```
ex:PersonShape a sh:NodeShape ;
  sh:targetNode ex:alice ;
  sh:property [ sh:path ex:name ; sh:minCount 1 ] .

[] a sh:ValidationResult ;
  sh:focusNode ex:alice ;
  sh:resultPath ex:name ;
  sh:sourceConstraintComponent sh:MinCountConstraintComponent .
```

If `ex:alice` has no `ex:name`, the result identifies the concrete failure directly. A `sh:node` result instead identifies failed conformance to another shape, whose concrete cause may be nested or absent.

2.3. Constraint Components and Leaf Failures

A result's `sh:sourceConstraintComponent` identifies the failed constraint [KK17; Lab+18]. The explainer treats `sh:NodeConstraintComponent` as a reference step rather than a final failure. All other constraint components are treated as leaf failures, including components outside the tested scope. Table 2.2 groups the Core components discussed here; Section 5.1.1 identifies which were actually tested beneath `sh:node`.

Table 2.2.: SHACL Core constraint components discussed in this thesis, with the short labels used for them. The explainer treats results with these components as leaf failures.

Constraint component	Label	Constraint
<code>sh:MinCountConstraintComponent</code>	<code>minCount</code>	At least a specified number of values.
<code>sh:MaxCountConstraintComponent</code>	<code>maxCount</code>	At most a specified number of values.
<code>sh:DatatypeConstraintComponent</code>	<code>datatype</code>	Values of a specified datatype.
<code>sh:ClassConstraintComponent</code>	<code>class</code>	Values are instances of a specified class.
<code>sh:PatternConstraintComponent</code>	<code>pattern</code>	Lexical forms match a regular expression.
<code>sh:InConstraintComponent</code>	<code>in</code>	Values belong to a specified set.
<code>sh:NodeKindConstraintComponent</code>	<code>nodeKind</code>	A specified RDF node kind.
<code>sh:MinInclusiveConstraintComponent</code>	<code>minInclusive</code>	Values greater than or equal to a bound.
<code>sh:MaxInclusiveConstraintComponent</code>	<code>maxInclusive</code>	Values less than or equal to a bound.
<code>sh:LessThanConstraintComponent</code>	<code>lessThan</code>	Values on one path less than those on another.
<code>sh:HasValueConstraintComponent</code>	<code>hasValue</code>	A specified value occurs.
<code>sh:EqualsConstraintComponent</code>	<code>equals</code>	Two paths have equal value sets.
<code>sh:DisjointConstraintComponent</code>	<code>disjoint</code>	Two paths have disjoint value sets.
<code>sh:ClosedConstraintComponent</code>	<code>closed</code>	Only properties permitted by a closed shape.
<code>sh:OrConstraintComponent</code>	<code>or</code>	At least one listed shape is satisfied.
<code>sh:XoneConstraintComponent</code>	<code>xone</code>	Exactly one listed shape is satisfied.
<code>sh:NotConstraintComponent</code>	<code>not</code>	A given shape is not satisfied.

`sh:NodeConstraintComponent` is excluded from the table because it represents `sh:n`ode, the reference expanded in the next section. If re-validation cannot break down a reference result without `sh:detail`, the explainer retains that result as a marked leaf (Section 4.6).

Scope. When `sh:node` occurs inside a logical or qualified constraint, pySHACL reports the enclosing component rather than a nested `sh:NodeConstraintComponent` result; the explainer therefore treats it as a leaf. Full support for those constraints and for complex property paths is outside the evaluated scope.

2.4. The `sh:node` Referencing Constraint

The `sh:node` constraint requires a node to conform to another shape [KK17; Lab+18]. A `sh:NodeConstraintComponent` result records failed conformance but may not reveal which constraint in that shape failed.

Two uses of `sh:node` must be distinguished because they determine which node is validated next.

Node-level `sh:node`. When declared on a node shape, the reference checks the same focus node against another shape. TC3 contains a two-step chain:

```
ex:ContractorShape sh:node ex:EmployeeShape .
ex:EmployeeShape sh:node ex:PersonShape .
ex:PersonShape sh:property [
  sh:path ex:age ; sh:datatype xsd:integer ] .
```

An invalid `ex:age` value fails the constraint in `ex:PersonShape`. The ordered route is a **reference chain**: `ex:ContractorShape -> ex:EmployeeShape -> ex:PersonShape`.

Property-level `sh:node`. When `sh:node` occurs on a property shape, every value reached through that property's `sh:path` must conform to the referenced shape. For example (fixture TC7):

```
ex:EmployeeShape
  a sh:NodeShape ;
  sh:property [
    sh:path ex:worksFor ;
    sh:node ex:CompanyShape
  ] .
```

```
ex:CompanyShape
  a sh:NodeShape ;
  sh:property [ sh:path ex:legalName ; sh:minCount 1 ] .
```

If `ex:alice ex:worksFor ex:acme`, then `ex:acme`, not `ex:alice`, is checked against `ex:CompanyShape`. Figure 2.1 summarizes the change of focus:

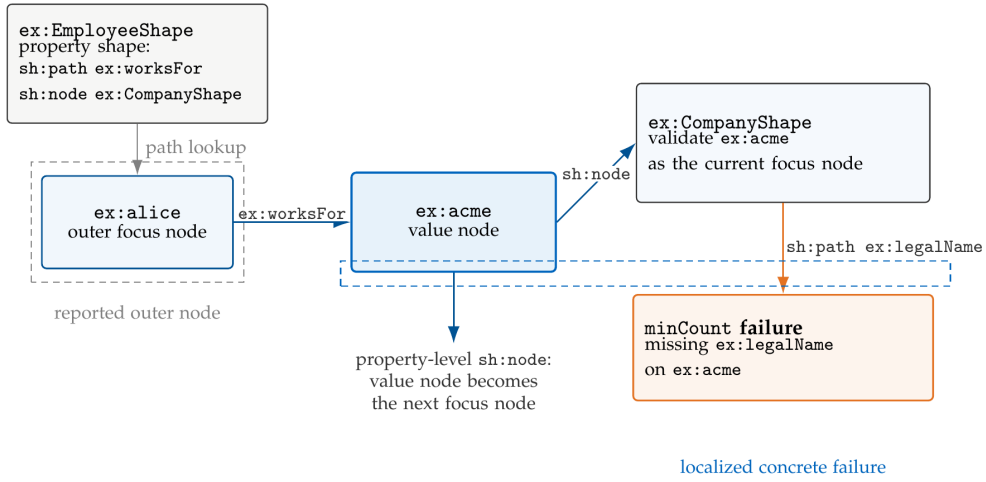


Figure 2.1.: Property-level `sh:node` in TC7: the employer value `ex:acme` becomes the focus node for `ex:CompanyShape`.

The outer TC7 result concerns `ex:alice` and `ex:worksFor`; the nested `minCount` failure concerns `ex:acme` and `ex:legalName`. Such nested results may be linked by `sh:detail` [KK17; Lab+18]. A chain can contain several reference results before a concrete failure, and two chains can reach the same failure. Chapter 4 describes their expansion and deduplication.

2.5. Terminology Used in This Thesis

These terms describe the explainer’s output; they are not additional SHACL constructs. Figure 2.2 shows them together. An **explanation tree** starts at the reported focus node. Inside the tree, each **reference node** represents one traversed `sh:node` step. For a property-level reference, it also records the path. A **leaf failure** represents a concrete constraint violation and records its path and value where available. Finally, the **reference chain** is the ordered sequence of shapes from the root to the leaf. A direct failure has an empty reference chain.

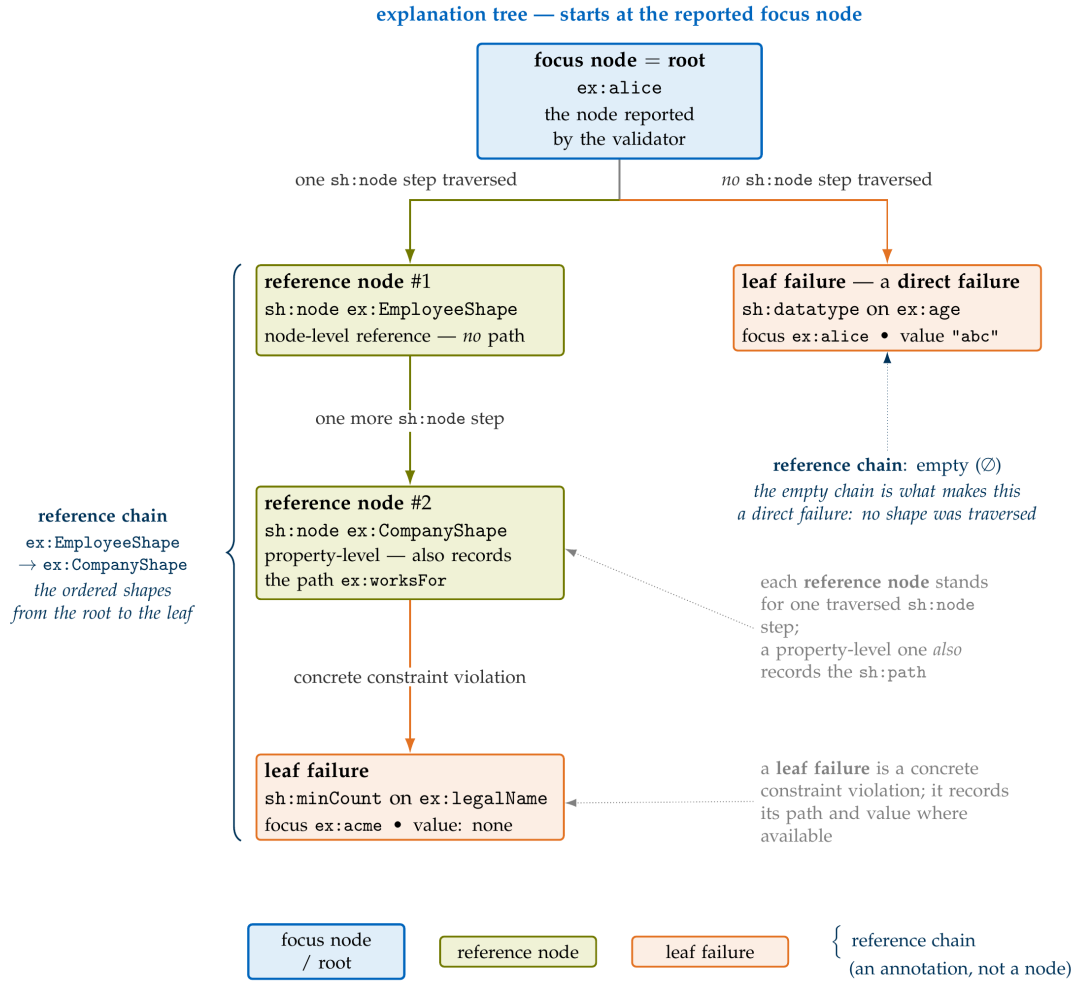


Figure 2.2.: Labelled explanation tree introducing the terminology of this thesis. The left branch traverses two references and therefore holds two reference nodes: a node-level one, which records no path, and a property-level one, which also records a path. The brace marks its reference chain, and the branch ends in a leaf failure on the value node `ex:acme`. The right branch traverses no reference, so its chain is empty and its leaf is a direct failure.

2.6. External Validation Reports and Optional `sh:detail`

Nested `sh:detail` results are optional in SHACL [KK17; Lab+18]. `pySHACL` can expose them for `sh:node`, while Apache Jena can report a reference failure without them [SC26; Apa26].

Property-level reports introduce another difficulty. Jena and TopBraid may identify the property shape using a fresh blank node [Top26]. This identifier cannot be matched directly to the corresponding property shape in the original shapes graph.

Other information may still be available, such as `sh:resultPath`, `sh:value` and the focus node. These fields can help determine what should be checked again. However, when metadata is missing, only part of the original reference chain may be recoverable. Section 4.6 gives the reconstruction procedure and its limits.

2.7. Recursive and Cyclic Shape References

Shape references can be recursive. A **self-recursive** shape references itself through `sh:node`; **mutual recursion** occurs when several shapes form a cycle such as `ex:ShapeA -> ex:ShapeB -> ex:ShapeA`.

SHACL leaves validation of recursive shapes to processor implementations [KK17, Section 3.4.3]; proposed formal semantics are discussed in the literature [PKM22; BJ21]. A cycle is not itself a failure. The branch-local guard in Section 4.4 lets explanation traversal terminate while preserving failures on other branches.

3. Related Work

SHACL reports identify failed constraints, but a failure involving `sh:node` may say only that a value does not conform to a referenced shape; the concrete cause lies inside that shape [KK17; Lab+18]. De Meester et al. find that conforming reports expose no root cause for nested shapes [De +21]. Report volume adds a second problem: Mäkelburg et al. describe a graph of more than 30 million triples whose validation produced over 10 million violations and a report larger than the graph [Mäk+25]. The relevant work therefore addresses both what an explanation contains and how a reader can navigate many results.

The comparison below groups approaches by their output: repairs, provenance and proofs; generated explanations and report exploration; and work on dependent shapes, recursion and scale. Table 3.1 summarizes the tool-level capabilities most relevant to this thesis.

3.1. Repairs, Provenance and Proofs

Ahmetaj et al.: repairs under minimality. Ahmetaj et al. formalize an explanation for SHACL non-validation as additions and deletions that make a graph satisfy its constraints. They study preferred explanations under subset- and cardinality-minimality, with complexity varying by task and restriction [Ahm+21]. Their Answer Set Programming implementation generates and optimizes repairs, including maximal repairs when not all targets can be repaired together; later work extends the setting to property paths and path equality [Ahm+22; Ahm+25]. The preference criteria matter because a graph can usually be repaired in several ways: a repair can be subset-minimal without using the fewest edits. Consistent query answering then asks which answers hold across minimal repairs [AMP24]. These approaches compute changes to, or conclusions over, repaired graphs. The local repair hints in this thesis do not carry a minimality guarantee; its primary output is the path from a reported `sh:node` failure to a concrete violated constraint.

Delva et al.: shape fragments. Delva et al. instead identify the data behind an outcome. Their *neighborhood* of a node for a shape contains triples relevant to conformance and satisfies a sufficiency property: keeping that neighborhood preserves conformance. Negated shapes provide why-not provenance, and unions of neighborhoods form shape

fragments for subgraph retrieval [Del+23]. An implementation adapting pySHACL reports average overhead below 10%, rising to 15.6% for shapes whose validation took over a second; its evaluated setting covers most of SHACL Core but assumes non-recursive schemas and excludes some complex path expressions [Del+23]. The neighborhood is deterministic even when several witnesses could satisfy a constraint. That property makes its output useful for reproducible subgraph extraction as well as explanation, while the sufficiency guarantee applies to conformance under the paper’s defined semantics. A neighborhood explains which triples matter. The tree in this thesis instead shows which constraint failed and the sequence of shape references leading to it; it makes no sufficiency claim about the data it displays and works after an existing validator has produced a report.

Validatrr: proofs from the reasoner. Validatrr is closer to the root-cause problem. De Meester et al. find that conforming reports fully explain the cause for 13 of the 28 SHACL Core components, identify a failing resource and constraint without a full cause for eight, and expose no root cause for seven, including nested shapes [De +21]. Validatrr translates validation into N3Logic and uses the EYE reasoner, whose proofs can expose the derivation of a violation. Validatrr can derive a proof across the components it covers, including nested shapes, because the reasoning system is responsible for validation itself [De +21]. This thesis likewise seeks the cause beneath a nested failure, but accepts an existing engine’s report and reconstructs its `sh:node` reference chain afterward. It therefore inherits the validator’s chosen result and cannot turn that result into a proof of the validator’s internal reasoning; its narrower claim is that a reported reference failure can be traced to concrete leaves when the report or a targeted re-validation supplies the needed evidence.

3.2. Generated Explanations and Report Exploration

xpSHACL: generated explanations. xpSHACL turns pySHACL violations into a justification tree, retrieves ontology and shape context, then uses a language model to generate an explanation and correction hint [PL25]. It caches explanations by violation signature in a Violation Knowledge Graph to limit repeated model calls and keep repeated descriptions consistent. On a 2 MB synthetic dataset, the authors report roughly 65 seconds for an uncached xpSHACL run against about 4 seconds for pySHACL, with cached runs stabilizing near 20 seconds; a separate experiment over LOV ontologies reports a cache hit rate near 99.5 percent [PL25]. Those figures illustrate the cost of adding retrieved context and model generation, although the systems were not benchmarked under the same protocol as this thesis. The authors also identify a formal user study as future work. Both xpSHACL and this thesis build structured

evidence, but xpSHACL uses the tree as input to generated prose, whereas this thesis presents a deterministic tree as the explanation. Reconstruction when `sh:detail` is missing and preservation of alternate diamond routes are not reported for xpSHACL.

SHACL Dashboard: report-wide aggregation. The SHACL Dashboard focuses on report-wide analysis. It aggregates violations by shape, property path, focus node and constraint component and supports drill-down to individual results [Mäk+26]. Its large-report demonstration illustrates why counts and distributions become important when millions of violations must be inspected [Mäk+25]. The Dashboard answers questions such as which shapes or paths account for most failures and lets a user move from a distribution to its contributing results. That is a different starting point from following one nested failure to its leaf cause.

SHACLens: coordinated views. SHACLens offers coordinated views of shapes, ontology structure, focus nodes and violations, including graph, hierarchy, table and dimensionality-reduction views alongside an LLM assistant [Ste+26]. Its evaluation observes that users can lose orientation between classes, shapes and violations and that one upstream data-quality issue can produce many downstream results. Its graph and hierarchy views help users navigate those relationships, while dimensionality reduction and tables help inspect large sets. The paper accordingly argues for more direct root-cause evidence across the report, a concern shared by this thesis even though the representations differ [Ste+26].

The HTML output in this thesis also supports navigation and aggregation, but its unit of explanation is a reconstructed failure: it follows `sh:node` references to a leaf before grouping the resulting failures. The Dashboard and SHACLens provide broader report analytics; neither publication reports rebuilding missing nested `sh:detail` or retaining several routes to one deduplicated leaf [Mäk+26; Ste+26]. SHACLens groups related violations, but that grouping is distinct from deduplicating the same leaf reached by several reference chains [Ste+26].

3.3. Dependent Shapes, Recursion and Scale

Trav-SHACL: planning over dependent shapes. Trav-SHACL optimizes validation over networks of dependent shapes. Its inter-shape planner orders evaluation across the schema dependency graph, while its intra-shape planner uses information learned during validation to improve queries; LUBM-based experiments report speedups of up to 28.93 over SHACL2SPARQL and earlier production of initial validation results [FRV21]. Its planning decisions use the schema’s reference structure to decide where validation effort is spent; this thesis uses that structure after validation to show how a failure was reached. Trav-SHACL produces sets of entities satisfying a schema rather

than the SHACL report consumed here, so combining the approaches would require a report interface that this thesis has not evaluated.

Formal semantics of references and recursion. Formal work clarifies what shape references mean, particularly when they recurse. Description-logic characterizations of SHACL, studies of recursive SHACL and broader foundations for recursive shape languages address validation semantics rather than the presentation of one validator’s outcome [BJV22; CRS18; Ahm+26]. SHACL leaves recursive validation to processor implementations [KK17, Section 3.4.3; see also PK22]. Shape Expressions (ShEx) also permits references between named shapes, such as a value required to satisfy another named shape, and handles recursion through inference rules that track already-assumed shape-value pairs [PLS14]. Revisiting an assumed pair in that proof system is structurally related to encountering a repeated node-shape pair in an explanation chain, but the language and the role of the tracked pairs differ. The cycle guard in this thesis merely ends a repeated branch while walking a report; it does not choose a semantics for recursive validation.

Practitioner evidence on scale. Empirical work motivates testing beyond small examples. In a practitioner survey, 45% of Rabbani et al.’s respondents worked with graphs above one million triples, while substantial proportions used tens or hundreds of shapes [RLH22]. The study also reports that shape creation remains largely manual and that some extraction tools struggle on graphs of only a few million triples. These observations motivate the LUBM scale experiment, but neither that survey nor the SHACL reviews establish how prevalent `sh:node` is in deployed schemas [PK22]. The large-graph evaluation here measures explanation cost and recurring failures for the supplied schemas; it does not estimate adoption of shape references in practice.

3.4. Comparison and Research Gap

Table 3.1 compares published tool capabilities; its structure follows the Dashboard comparison [Mäk+26]. A *not reported* entry is not a negative finding. Repair generation also varies: ASP repairs satisfy formal preference criteria, xpSHACL and SHACLens generate hints, and this thesis attaches local heuristics to supported leaves [Ahm+22; PL25; Ste+26].

3. Related Work

Table 3.1.: Comparison of this thesis with closely related tools. *Not reported* means that the cited work does not discuss the capability; *n/a* means that the system does not consume an existing SHACL validation report.

System	Reference chains	Without <code>sh:detail</code>	Rule-based, no LLM	Repair hints	Diamond deduplication	Interactive views	Report-wide analysis
This thesis	yes	yes	yes	yes, heuristic	yes	yes	yes, within one report
xpSHACL [PL25]	not reported	not reported	no	yes, generated	not reported	no	no
ASP repairs [Ahm+22]	not reported	n/a	yes	yes, minimal	not reported	no	no
Shape fragments [Del+23]	not reported	n/a	yes	no	not reported	no	no
Validatrr [De+21]	yes, via proof	n/a	yes	no	not reported	no	no
Trav-SHACL [FRV21]	not reported	n/a	yes	no	not reported	no	no
SHACL Dashboard [Mäk+26]	not reported	not reported	yes	no	not reported	yes	yes
SHACLens [Ste+26]	not reported	not reported	no	yes, generated	not reported; comparable grouping	yes	yes

Across these systems, the surveyed literature does not report the particular combination developed here: reconstructing `sh:node` chains from an existing validation report, recovering nested failures when `sh:detail` is absent, and deduplicating a leaf while retaining its alternate reference routes. This is a claim about what the cited publications report, not about every possible implementation. Chapter 4 presents the reconstruction procedure.

4. Our Approach

Core idea. A SHACL validation report gives the authoritative validation outcome, but it does not always give a repair-oriented explanation of that outcome. When a focus node fails a referencing constraint, the report shows that it does not conform to the referenced shape. However, the concrete violation that caused the failure may be nested under `sh:detail` or missing entirely. The approach presented here treats the validation report as an input to be post-processed rather than as the final explanation. The tool constructs an *explanation tree* from either a pySHACL report [SC26] or an external validation report. Its interior nodes represent traversed `sh:node` references. Its leaves represent concrete SHACL Core violations and record the constraint component, path, value where available, and reference chain (Figure 2.2). For constraint types where a safe local hint can be derived, the presentation layer also attaches a repair hint.

The approach is deliberately narrower than a new SHACL validator or repair system. Validation remains the responsibility of an existing SHACL engine; the contribution is the reconstruction and organization of the evidence it reports. Two cases require different treatment. If nested results are present as `sh:detail`, explanation construction is a recursive traversal of the report graph. If they are missing, the tool reconstructs the missing evidence by re-validating only the relevant node against a targeted copy of the referenced shape. After expansion, duplicate leaves created by diamond-shaped references are collapsed while their alternate reference chains are retained.

A second design principle is that the explanation tree is the shared representation for all later operations. Deduplication, filtering, serialization and visualization operate on the same node types rather than implementing separate interpretations of the report. This keeps the semantic reconstruction independent of the output format and allows the same explanation to be inspected as text, structured data, aggregate statistics or an interactive report.

4.1. Pipeline Overview

The pipeline must preserve the report’s hierarchy while making its concrete failures usable in several output modes. Figure 4.1 shows its five stages: the command-line layer loads the graphs and either runs pySHACL or accepts an existing report (1); the parser selects the direct `sh:result` roots (2); the expander expands references into

explanation nodes, reconstructing missing detail through the fallback branch where necessary (3); the deduplicator collapses duplicate leaves while retaining their routes (4); and the renderer writes the resulting tree in the selected output mode (5).

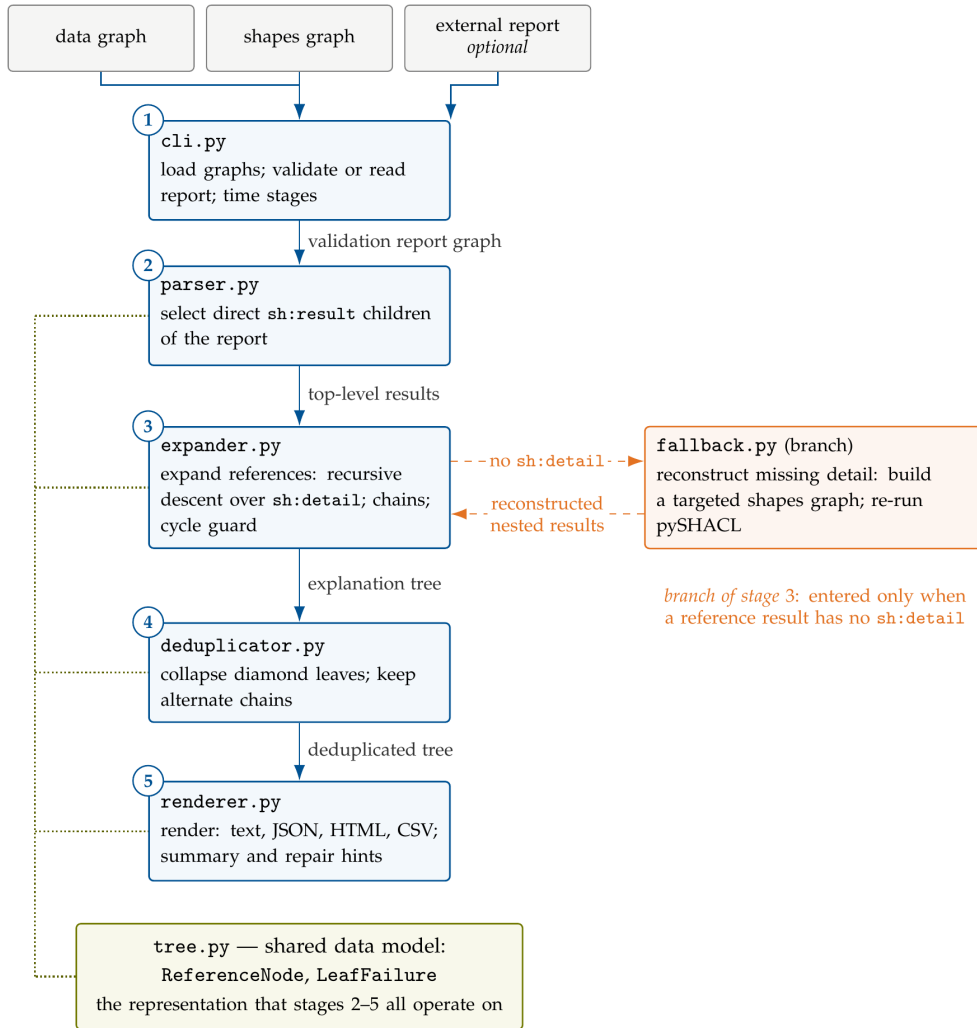


Figure 4.1.: Architecture of the explainer: the five stages of the pipeline. The fallback branch inside stage 3 is taken only when a `sh:NodeConstraintComponent` result does not provide nested `sh:detail` results.

The implementation files and functions for these stages are listed in Appendix A. The separation between construction and presentation is important. The expander determines what a failure means. The renderer only decides how that already-constructed

explanation is shown. Consequently, switching from text to JSON or HTML does not change root selection, reference-chain reconstruction or deduplication. This distinction also makes the contribution easier to evaluate: Chapter 5 can measure tree construction separately from validation and rendering.

4.2. Extracting Top-Level Results

Nested details are also `sh:ValidationResult` nodes. Therefore, selecting every node of that type would flatten the report hierarchy. The same concrete violation could then be counted both as a root and as a descendant. The rule is to use only results linked directly from `sh:ValidationReport` by `sh:result` as explanation roots; `sh:detail` results remain below their parent. In TC2, pySHACL produces four validation-result nodes, but only one is a direct report child. Direct Core failures and top-level referencing failures then enter expansion through the same root list.

Keeping parsing this small is intentional. The parser does not interpret property shapes, infer referenced shapes or reconstruct missing detail. Those decisions belong to expansion and fallback, where both the report graph and the shapes graph are available.

4.3. The Explanation-Tree Data Model

Later stages need a shared account of each failure and the route that reached it. The concepts introduced in Section 2.5 are realized here by two node types: reference steps and concrete failures. Table 4.1 summarizes their fields.

Table 4.1.: The two node types used by the explanation tree.

Node	Represents	Essential fields
ReferenceNode	One traversed <code>sh:node</code> step	focus node, displayed/source shape, referenced shape, reference path, message, reference chain, children
LeafFailure	One concrete constraint violation	focus node, path, component, value, source shape, message, reference chain, alternate chains

A `ReferenceNode` preserves the hierarchical part of the explanation. Its children may be further references or concrete leaves, so arbitrary nesting can be represented directly.

A `LeafFailure` represents the point at which expansion has reached a non-referencing constraint. Every node therefore stores its own focus node instead of inheriting the root's focus node. This is necessary because a property-level reference can shift the focus to another node (Section 2.4). The deduplication identity described below uses the leaf's own focus node.

Both node types carry the reference chain accumulated at the point where they are created. The chain is stored explicitly rather than reconstructed from parent pointers. This makes each leaf self-describing: text output can print the route beside the failure, CSV can store it in a row, JSON can expose it directly and the summary can aggregate chains without traversing upward through the tree. `LeafFailure` additionally stores alternate chains discovered during deduplication, allowing several routes to one violation to be preserved without duplicating the violation itself.

The data model also separates semantic identity from presentation. A leaf is identified by its focus node, result path, constraint component and value node; the reference chain is excluded because different chains may reach the same failure. Human-readable labels, URI shortening and repair wording are added later. This prevents formatting choices from affecting whether two report entries are considered the same underlying failure.

The intermediate representation therefore does more than simplify serialization. It separates interpretation of the validation report from later transformations of the tree.

4.4. Recursive Expansion

One validation result can describe either a concrete failure or a reference to another shape, so expansion must distinguish them before descending. The explainer checks the result's `sh:sourceConstraintComponent`. A `sh:NodeConstraintComponent` indicates another reference step, so the branch is expanded further. Any other component is treated as a leaf failure, including logical, qualified and SHACL-SPARQL components. Table D.2 records which components the fixtures tested. A cycle guard may stop a reference branch without producing a node.

For a concrete result, the explanation records the focus node, result path, value, component, message and source shape. Where a shape supplies several `sh:resultMessage` values, all are sorted and joined to give a stable explanation. Validation engines can identify an anonymous property shape as the source, which would make the route difficult to read. Where an enclosing node shape can be found, the display uses that shape name while the leaf retains the reported path and component. Thus a chain such as `ex:ContractorShape -> ex:EmployeeShape -> ex:PersonShape` ends in a recognizable shape without changing the failure itself.

For a referencing result, the expander creates a `ReferenceNode` and determines its children. If `sh:detail` is present, each nested result is recursively expanded. The current display shape is appended to the reference chain before descending, so a leaf reached after several references receives the complete route that led to it. The report graph remains the primary source of result-level information, while the shapes graph supplies shape structure that an external report may name but not embed.

The recursion must also preserve branches independently. A single result can contain several detail results, and different branches may legitimately reach the same shape. Chains are therefore extended by creating a new list for each recursive call rather than mutating a shared list. The same principle is used for cycle detection.

Guarding against cycles. As discussed in Section 2.7, recursive shape references can return to a shape already encountered on the same route. The mechanism that stops this, the cycle guard, works as follows. A naive recursive traversal would have no structural stopping condition. The expander therefore tracks visited pairs of focus node and shape. Before descending through a reference, it checks whether that pair has already occurred on the current branch. If so, that branch stops. Otherwise the pair is added to a new branch-local visited set and expansion continues.

The guard is branch-local rather than global. This distinction is essential: a global visited set would incorrectly suppress a second, legitimate route in a diamond-shaped graph simply because another branch had already reached the same shape. A branch-local set stops `A -> B -> A` while still allowing sibling paths `A -> B -> D` and `A -> C -> D` to both reach `D`. The visited key intentionally uses the focus node and shape rather than the property path, so returning to the same validation state through a different property does not evade the guard. TC8 exercises this case: `pySHACL` produces a cyclic set of results reached through different property paths, while the explanation traversal terminates and retains the concrete `ex:required` leaf.

`build_explanation_tree` starts with the parser's roots and expands them into the explanation tree. Branches stopped by cycle detection are omitted before the complete tree is passed to deduplication. Before expansion, `sort_results` puts the top-level results into a fixed order: referencing results first, followed by direct failures, then focus node, path, constraint component and value; source shape and message break ties. Validator output and `rdflib` iteration are not stable across runs because Python reseeds string hashing for each process. Blank-node labels also change when a graph is parsed, so blank nodes are ordered by the triples they carry instead. The same order is used for the `sh:detail` children of each result and for results produced by re-validation. This does not change which failures the tree contains. It determines which root is processed first when several roots reach the same leaf and which reference chain is kept as primary (Section 4.7). Fixing the order therefore makes the output identical on every run.

Two aspects of this procedure make the tree independent of a particular validator's report style. First, classification relies on the reported constraint component rather than on whether `sh:sourceShape` is named or blank. Second, shape metadata can be resolved from both the report and the original shapes graph. `pySHACL` often carries enough structure in its report for a direct descent, whereas external reports may only identify a source shape and path. The expander therefore uses the report where it is informative but does not assume the report contains a copy of the shapes graph.

4.5. Node-Level Versus Property-Level References

In TC7, `ex:alice` reaches `ex:acme` through `ex:worksFor`, and `ex:acme` fails `ex:Company Shape`. Section 2.4 distinguished property-level from node-level `sh:node`; the expansion algorithm must preserve that distinction because it determines which node is explained and, when detail is missing, which node is re-validated.

For a node-level reference, the referenced shape is evaluated against the same focus node. The explanation therefore keeps that node unchanged while extending the chain of shapes. For a property-level reference, the referenced shape is evaluated against a value reached through the property shape's `sh:path`. The outer result's path identifies how the new node was reached, and its `sh:value`, when present, becomes the focus node for the nested explanation. The resulting leaf is consequently attributed to the value node rather than to the original root focus node.

A property-level reference displays the referenced shape and its path; a node-level reference displays the shape declaring the step. Both still record the referenced shape separately, and chain depth counts `sh:node` steps rather than displayed names. Table 4.2 summarizes the rule; the TC7/TC3 text examples are in Appendix B.

Table 4.2.: Property-level versus node-level display rules.

Aspect	Property-level (TC7)	Node-level (TC3)
Focus node	Shifts: the value reached through the property's <code>sh:path</code> (<code>ex:acme</code> , reached from <code>ex:alice</code>) becomes the new focus node.	Unchanged: the outer root focus node (<code>ex:carol</code>) carries into every nested shape.
Shape shown at the step	The shape <i>referenced</i> by <code>sh:node</code> , i.e. the target the value is checked against (<code>ex:CompanyShape</code>).	The shape that <i>declares</i> the <code>sh:node</code> reference, i.e. the source of the step (<code>ex:ContractorShape</code> , then <code>ex:EmployeeShape</code>).
Step written as	<code>ex:EmployeeShape --sh:node[path=ex:worksFor]--> ex:CompanyShape</code> ; the path is carried in the arrow.	<code>ex:ContractorShape --sh:node--> ex:EmployeeShape, then ex:EmployeeShape --sh:node--> ex:PersonShape</code> .
Chain	One displayed shape per traversed step, followed by the shape owning the leaf's constraint when it is not already the last one shown. Here the owning shape is <code>ex:CompanyShape</code> , which the step already displays, so nothing is appended: <code>ex:CompanyShape</code> , one name for one step.	The same rule. Here the owning shape, <code>ex:PersonShape</code> , is not yet shown, so it is appended: <code>ex:ContractorShape -> ex:EmployeeShape -> ex:PersonShape</code> , three names for two steps.
Depth	The number of <code>sh:node</code> steps traversed, which is 1. The focus-node shift happens at that step, not at an extra one.	The same rule, which is 2.

The terminal renderer prints one shape name at each reference step, so the same `sh:node` label has a different role in the two cases. The appendix examples use arrows naming both ends solely to make the contrast explicit. The referenced shape is stored separately from the displayed label. If a source shape declares several `sh:node` references, the expander uses the `sh:sourceShape` values of the result's own `sh:detail` children to identify the reference whose failures the result contains. If those children are absent and several candidates remain, the field is left empty rather than filled with a guess. Deduplication can subsequently combine several node-level steps into one. The combined step retains one referenced-shape value even when its children were reached through different references; Section 5.4 discusses this remaining ambiguity. The display rule here describes the supported traversal, not a guarantee that every ambiguous source report identifies one unique reference.

This distinction is especially important when `sh:detail` is absent. For a property-

level result, re-validating the original focus node would reconstruct the wrong problem; the target must be the reported value node. The fallback procedure therefore uses the same node-transition rule as ordinary expansion, so direct traversal and reconstructed traversal yield the same kind of explanation node.

4.6. External Reports and the Fallback Path

An **external report** is a SHACL validation report supplied to the tool through `--report` rather than produced by the default pySHACL run. The report remains the authoritative statement of the validator’s outcome; the explainer only reorganizes or reconstructs evidence needed to expose the concrete cause.

The fallback path addresses the case in which such a report contains a `sh:NodeConstraintComponent` result with no nested `sh:detail`. The SHACL specification does not require `sh:detail` [KK17; Lab+18], and external validators may therefore report that a node does not conform to a referenced shape without embedding the concrete violations that caused that non-conformance. Apache Jena and TopBraid are two relevant examples [Apa26; Top26]. In such a report there is no child result for the expander to traverse, so explanation construction must recover the missing evidence from the data and shapes graphs. On the command line, this path is reached by supplying `--report` instead of running pySHACL directly: the tool loads the existing report and skips top-level validation, allowing an external validator’s report to enter the same expansion pipeline and, where required, the fallback reconstruction described below.

For TC7, reconstruction must recover `ex:CompanyShape`, select `ex:acme` rather than `ex:alice` as the target, and restrict re-validation to that target. These are the three problems the fallback solves for any report without usable detail.

Appendix B compares the incomplete-report variants and their fixtures. The following recovery rules apply to each variant.

Recovering the referenced shape. A report may name the enclosing node shape while the `sh:node` declaration sits on one of its property shapes. The rule first looks for `sh:node` on the reported source shape; otherwise it finds a property shape whose `sh:path` matches the reported path and reads the reference there. In the synthetic TC7 report without `sh:detail`, `sh:sourceShape` names `ex:EmployeeShape` and `sh:resultPath` names `ex:worksFor`; the shapes graph identifies `ex:CompanyShape` as the referenced shape.

Apache Jena and TopBraid usually identify the property shape that produced a property-level result. If the property shape is named, the tool can find it directly in the shapes graph. However, property shapes are commonly anonymous. In that case,

the report contains a new blank-node identifier that does not match the identifier in the original shapes graph. The tool therefore searches the shapes graph for property shapes that declare `sh:node` on the reported path. If several candidates use the same path, it narrows them using targeting information, validation evidence, severity and message metadata.

1. **Targeting.** Keep candidates whose property shape, or owning node shape, targets the reported focus node.
2. **Evidence.** Keep candidates for which the reported value fails the referenced shape. If the value conforms to a candidate's referenced shape, that candidate cannot have produced the violation.
3. **Metadata and pairing.** If several candidates remain, match them to results by declared `sh:severity` or by an exact `sh:message` match. If ambiguity remains and the numbers of results and candidates are equal, pair them one to one. Reference steps do not display these messages, so the pairing order does not affect the output.

Each filter is applied only if it leaves at least one candidate. Messages are used only for exact matching: the algorithm never interprets a `sh:resultMessage` or tries to extract a shape name from it. Messages are optional and validator-specific, while the shapes graph states the reference explicitly. Section 5.4 describes the case in which this narrowing was necessary and the ambiguity that can remain.

Choosing the target node. The node supplied to re-validation follows the same rule as normal `sh:node` semantics. A node-level reference applies the referenced shape to the focus node, which is therefore the target. A property-level reference applies it to the values of the path, so the reported `sh:value` is the target. A report may omit `sh:value`; the values are then found by following the reported path from the focus node in the data graph (`values_along_path`, which evaluates complex paths with pySHACL's own path evaluation), and each is re-validated. The focus node is never substituted for a missing value, since it is not the node the constraint was checked on. This ensures that a missing property on a referenced company, for example, is explained as a failure of that company rather than incorrectly attached to the employee whose `ex:worksFor` property led to it.

When re-validation finds nothing. Re-validation can come back empty although the validator reported a violation: the two validators may disagree, for instance on a recursive shape or on a feature pySHACL evaluates differently, or the path may lead to no value at all. The validator's own result is then kept, as a leaf carrying the `sh:NodeConstraintComponent` result and a note that says which of the two happened, and the

summary counts such leaves separately. An explanation therefore never loses a failure the validator reported, even where it cannot break that failure down.

Constructing a targeted shapes graph. Re-running the complete shapes graph could validate unrelated targets and reproduce much of the original report. The rule copies the referenced shape and the constraints needed below it into a smaller graph, then binds that shape to one node with `sh:targetNode`. Figure 4.2 applies the rule to TC7: the copy retains `ex:CompanyShape` and its property constraint, bound to `ex:acme`; it leaves behind `ex:EmployeeShape` and its target declaration. The data graph remains unchanged.

(a) Original shapes graph	(b) Targeted shapes graph
<pre> ex:CompanyShape a sh:NodeShape ; sh:property [sh:path ex:legalName ; sh:minCount 1 ; sh:message "ex:legalName is required"] . ex:EmployeeShape a sh:NodeShape ; sh:targetClass ex:Employee ; sh:property [sh:path ex:worksFor ; sh:minCount 1 ; sh:node ex:CompanyShape ; sh:message "ex:worksFor must point to a valid company"] . </pre>	<pre> ex:CompanyShape a sh:NodeShape ; sh:targetNode ex:acme ; sh:property [sh:path ex:legalName ; sh:minCount 1 ; sh:message "ex:legalName is required"] . </pre>

Figure 4.2.: TC7 shapes graph before and after targeted reconstruction. The copy retains `ex:CompanyShape` with its `ex:legalName` constraint and adds `sh:targetNode ex:acme`. The outer `ex:EmployeeShape` and its target declaration are omitted; the data graph is unchanged. Both graphs are the tool’s own triples, serialized as Turtle with the statement order and line breaks adjusted to fit the column.

In TC7, the copy starts at `ex:CompanyShape` and follows every edge its constraint needs in order to mean what it meant in the original. The following rules state what is copied and omitted beyond this small example.

What is copied.

- Every triple of the referenced shape except its target declarations. This includes its type, constraint parameters and `sh:property` links.
- Every blank node reachable below the referenced shape, at any depth. This includes anonymous property shapes, paths such as `sh:inversePath` and `sh:alt`

alternativePath, and the complete RDF lists used by `sh:in`, `sh:or`, `sh:and`, `sh:xor` and `sh:languageIn`. Thus, an `sh:in` retains all permitted values and an `sh:or` retains all branches.

- Every referenced named shape, followed recursively at any depth and through any predicate. This includes named property shapes linked by `sh:property`, node-level and property-level `sh:node` references, and shapes used in `sh:or` or `sh:xor` branches. A chain of named shapes is followed to its end.
- The shapes graph's prefix bindings. These allow pySHACL to use the original term names in messages for the fresh results.
- One new `sh:targetNode` triple that binds the copied root shape to the node being reconstructed.

What is not copied.

- Target declarations on the root shape or any copied shape: `sh:targetClass`, `sh:targetNode`, `sh:targetSubjectsOf`, `sh:targetObjectsOf` and `sh:target`. An implicit class target is also removed by omitting the shape's `rdfs:Class` or `owl:Class` type while retaining its `sh:NodeShape` type. Keeping these targets would validate unrelated nodes.
- Anything unreachable from the referenced shape. This excludes the enclosing shape, its other property shapes and every unrelated shape. For example, Figure 4.2 leaves out `ex:EmployeeShape`.
- Descriptions of non-shape terms named by constraints, such as the class used by `sh:class` or the datatype used by `sh:datatype`. The constraint still contains the term itself, while the unchanged data graph determines whether a value belongs to that class or datatype.
- A shape already copied during the same traversal. This recursion guard bounds the copy when the shapes graph is cyclic.
- Dependencies outside the SHACL Core structures followed by these rules. For example, a `sh:sparql` constraint is copied as a blank node, but the `sh:declare` block reached through `sh:prefixes` is not because it is an ontology rather than a shape. SPARQL-based constraints are outside the scope of this work (Chapter 1).

A complete recursive copy is necessary to preserve the validation result. An `sh:in` list copied without its members is empty, so every value fails it. A referenced shape

that is not copied has no constraints, so every value passes it. Section 5.3.2 reports the fixtures that test both cases.

The added `sh:targetNode` limits re-validation to one referenced shape and one node instead of all normal targets. The unchanged data graph still supplies the RDF facts. The local pySHACL run disables inference and collects all failures rather than stopping at the first.

The fresh report is processed like ordinary nested detail. Its top-level results return to the recursive expansion procedure with the existing reference chain. Whether supplied as `sh:detail` or reconstructed by fallback, each nested result becomes the same `ReferenceNode/LeafFailure` structure. If several re-validations are required, one per referenced shape or value, their results together replace the missing `sh:detail` and are ordered as described in Section 4.4.

Sharing re-validations. Different report results may require the same node to be checked against the same referenced shape. For example, hundreds of people may name the same university as their degree-granting institution. Without caching, the fallback would re-validate that university against the same shape hundreds of times.

During one call to `build_explanation_tree`, `RevalidationCache` stores the fresh report for each pair of target node and referenced shape. The first occurrence performs the re-validation, while later occurrences reuse the stored report. The report is only read and never modified, so reuse does not change the explanation. A test requires cached and uncached builds to produce the same output. Section 5.3.2 shows that the cache avoids 50–95% of re-validations on the LUBM reports.

In short, the procedure recovers the referenced shapes R and target nodes T , re-validates each shape–target pair with a targeted shapes graph S' , and returns the reconstructed results for normal expansion.

Procedure 2 Reconstructing a `sh:node` result without `sh:detail`

Require: A `sh:NodeConstraintComponent` result r without `sh:detail`

Require: The data graph D

Require: The shapes graph S

Ensure: Ordered reconstructed results, or a marked leaf if reconstruction finds none

- 1: $R \leftarrow$ the shapes named by `sh:node` on r 's reported source shape in S .
 - 2: **if** R is empty **then**
 - 3: Find property shapes owned by the source shape whose `sh:path` matches r 's reported path.
 - 4: Add the shapes named by their `sh:node` values to R .
 - 5: **end if**
 - 6: **if** R is empty **then**
 - 7: Find all property shapes in S that declare `sh:node` on the reported path.
 - 8: Narrow them using the targeting, evidence and metadata rules above; add their `sh:node` values to R .
 - 9: **end if**
 - 10: **if** R is empty **then**
 - 11: $R \leftarrow \{\text{the displayed shape of } r\}$ ▷ preserve an unresolved reference
 - 12: **end if**
 - 13: **if** r is node-level **then**
 - 14: $T \leftarrow \{\text{the focus node}\}$
 - 15: **else if** r carries a value **then**
 - 16: $T \leftarrow \{\text{the value node}\}$
 - 17: **else**
 - 18: $T \leftarrow$ the values reached from the focus node along r 's path in D
 - 19: **end if**
 - 20: Let E be an empty collection of reconstructed results.
 - 21: **for** each $s_{ref} \in R$ and each target $t \in T$ **do**
 - 22: Build S' by copying s_{ref} and all recursively required shape structures described above, but no target declarations.
 - 23: Add `sh:targetNode` t to S' so the copied shape is evaluated only on that node.
 - 24: Obtain a fresh report by running pySHACL on D and S' , reusing the cached report when this pair was already checked.
 - 25: Add the fresh report's top-level results to E .
 - 26: **end for**
 - 27: **if** E is empty **then**
 - 28: **return** a marked leaf based on r , with a note that it could not be broken down
 - 29: **end if**
 - 30: Put all results in E into the fixed order described in Section 4.4.
 - 31: **return** E for expansion as children of the current reference node with Procedure 1.
-

Figure 4.3 shows the same reconstruction as a data flow. The two recovery steps are what the absent `sh:detail` would otherwise have supplied; the targeted shapes graph is what keeps each re-validation local, and one such validation is run for every shape–target pair.

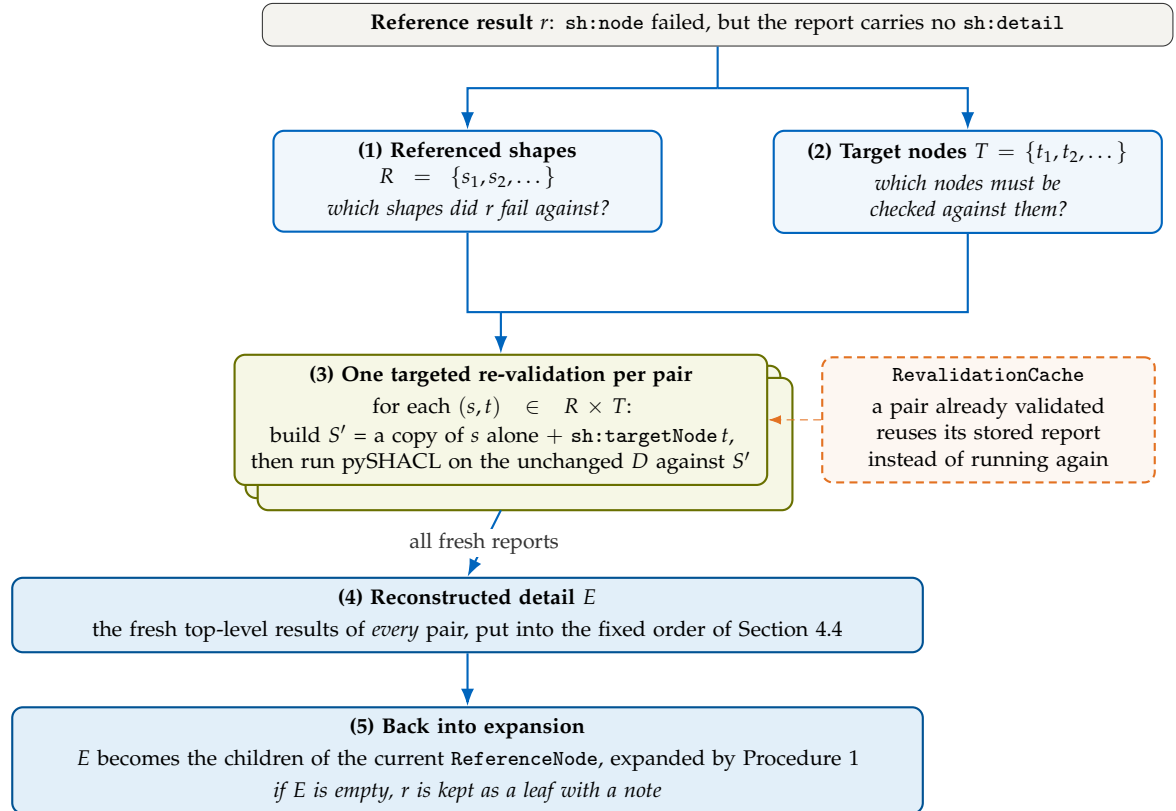


Figure 4.3.: Targeted re-validation when an external report omits `sh:detail`. Steps 1 and 2 recover what the missing nested result would have carried, by the search rules of Procedure 2. Because both are sets, step 3 runs one targeted validation per shape–target pair, each restricted to a single node and a single referenced shape, and the cache removes repeated pairs. The results of all pairs together replace the missing `sh:detail`.

The targeted copy has a separate recursion guard. If one referenced shape eventually points back to a shape already copied, the operation could otherwise continue indefinitely. It therefore records each copied shape and stops when it reaches the same shape again. This prevents construction of the temporary fallback graph from looping before validation begins.

The copy has no fixed maximum depth. It follows reachable blank nodes and named shapes until no new structure remains. Its size therefore depends on how densely the shapes refer to one another. The schemas evaluated in Chapter 5 use such cross-references only sparsely.

A direct SHACL Core failure can still be represented when `sh:sourceShape` is missing. Its focus node, path, component and message are sufficient to create a leaf. Because the source shape is unknown, the tool does not invent a reference chain. Appendix B shows the TC45 output.

Some information cannot be reconstructed. If missing shape information would have established a reference chain, that chain remains unknown. Likewise, if a `sh:node` result does not contain enough structure to identify the referenced shape, the tool cannot guarantee the same explanation as a complete nested report.

The TC2 and TC5 fallback tests check cases in which reconstruction should succeed. They remove every `sh:detail` triple from complete pySHACL reports and verify that the expected leaf failures are recovered. The TC5 test also requires both routes through its diamond-shaped reference structure. These tests demonstrate successful reconstruction under controlled conditions, but they do not show that every incomplete external report can be fully reconstructed. Chapter 5 presents this limitation explicitly rather than inferring missing structure.

Why re-validation is targeted rather than global. Re-running the original shapes graph would be simpler conceptually, but it would weaken both efficiency and explanatory locality. The original validation may contain many targets unrelated to the current reference result, and a full second run would regenerate those failures before the tool could identify which belong below the current node. By copying only the referenced shape and binding it to one `sh:targetNode`, the fallback asks the validator the narrower question the missing `sh:detail` would have answered: why does this particular node fail this particular referenced shape? The resulting report is therefore small enough to be attached directly beneath the current reference step.

The fallback also preserves validator authority. The tool does not implement each SHACL Core component itself in order to infer what must have failed. It delegates the missing local validation to pySHACL and then interprets the returned standard report using the same expansion procedure as before. This reduces the risk that the reconstruction logic and the validator implement subtly different constraint semantics. The trade-off is an explicit dependency on pySHACL for reconstruction even when the top-level report originated from another validator.

This fallback is the part of the approach that most clearly separates explanation reconstruction from simple report formatting. A renderer could expose existing `sh:detail`, but it could not explain a validator report in which the detail never existed. Targeted re-validation supplies that missing layer while keeping the reconstructed

results in the same deterministic tree representation used for native reports.

4.7. Deduplicating Diamond-Shaped References

After TC7 is expanded or reconstructed, its `ex:legalName` failure has one reference chain and remains one leaf. A different problem appears when the same concrete failure is reachable through several valid chains. This thesis calls that shapes-graph pattern a **diamond reference**: two branches converge on the same downstream shape. A validator can consequently report the same leaf constraint once along each route. Showing all copies would exaggerate the number of distinct data problems; discarding all but one route would lose information about how the schema reached the problem. **Deduplication** collapses repeated copies while preserving the extra routes as **alternate chains**.

The rule identifies a concrete failure by four values: focus node, result path, constraint component and value node. These fields describe what failed and where; the reference chain is excluded because a diamond creates different routes to the same failure. During depth-first traversal, the first leaf with an identity survives, while later distinct non-empty chains become its alternate chains.

TC7 cannot show a diamond because it has only one route to `ex:CompanyShape`. TC5 supplies this second case in Listing 3: `ex:StaffShape` reaches `ex:PersonShape` through both a full-time and a part-time branch. The `ex:age` and `ex:name` violations appear once each, retaining the second route as an alternate chain. Branch-specific failures remain separate because their paths differ.

```
Focus node: ex:eve/
+-- sh:node ex:StaffShape/
  |-- sh:node ex:FulltimeShape/
  | |-- [failure] datatype ex:age = "not-a-number"
  | | |-- via ex:StaffShape -> ex:FulltimeShape -> ex:PersonShape
  | | |-- message: ex:age must be an xsd:integer
  | | +-- also via ex:StaffShape -> ex:ParttimeShape -> ex:PersonShape
  | +-- [failure] minCount ex:name = (missing)
  | |-- via ex:StaffShape -> ex:FulltimeShape -> ex:PersonShape
  | |-- message: ex:name is required
  | +-- also via ex:StaffShape -> ex:ParttimeShape -> ex:PersonShape
  |-- [failure] minCount ex:contractHours = (missing)
  | |-- via ex:StaffShape -> ex:ParttimeShape
  | +-- message: Part-time staff must declare contractHours
  +-- [failure] minCount ex:hoursPerWeek = (missing)
    |-- via ex:StaffShape -> ex:FulltimeShape
    +-- message: Full-time staff must declare hoursPerWeek
```

Listing 3: Deduplicated explanation of the TC5 diamond. Shared failures appear once while both routes remain explicit.

Deduplicating leaves can create empty reference branches. If every child of a reference node was a duplicate moved onto another surviving branch, that reference node no longer carries a distinct failure and is removed. Repeated sibling reference nodes representing the same step are merged with their surviving children, then ordered by content so the result does not depend on which sibling came first. These operations keep the output tree compact without changing any remaining leaf or alternate route.

The transformation is non-mutating, so the same expanded tree can be reused for more than one operation and applying deduplication twice has the same result as applying it once.

The chosen deduplication key is intentionally conservative. It does not attempt semantic equivalence between arbitrary SHACL constraints or RDF terms. Two results are merged only when they identify the same focus node, path, component and value according to the report representation. This is sufficient for the diamond patterns evaluated in this thesis and avoids introducing an additional reasoning layer whose correctness would be harder to justify.

The source shape is not part of this identity. Therefore, constraints declared in different shapes are merged when they produce the same component, path, focus node and value.

Deduplication also operates across all top-level roots. `deduplicate` uses one `seen_leaves` map for the complete forest. If several roots reach the same leaf, the leaf is kept under the first root in the fixed order of Section 4.4. Later routes may be stored as alternate chains, but the other roots are not recorded. A root is removed if all of its children disappear during this process.

The output therefore counts distinct concrete failures rather than all reported results. A future version could include the source shape in the identity and record every root that reaches a leaf. This would preserve more provenance and would change the failure and root counts. Section 5.4 explains how the current design affects the reported counts.

Reference nodes require a related cleanup after leaf collapse. Suppose two reported branches encode the same focus node, source shape and incoming reference chain. If their children are merged only at the leaf level, the final tree can still contain duplicate intermediate reference nodes. The deduplicator therefore combines equivalent sibling reference nodes and concatenates their surviving children. It also removes a reference node whose children all disappeared as duplicates. This cleanup is structural: it does not create new routes or alter the identity of any leaf.

Two aspects of Listing 3 are easy to misread in isolation, because two mechanisms act on the tree at once and neither leaves a visible trace in the final text. First, `ex:hoursPerWeek` is `ex:FulltimeShape`'s own property, yet it sits beside the `sh:node`

4. Our Approach

`ex:FulltimeShape` node rather than inside it: node-level display always names the *declaring* shape (Section 4.5), so the node still labelled `ex:StaffShape` is really the "StaffShape references FulltimeShape" step, and everything `sh:detail` attaches to that step, including `ex:FulltimeShape`'s own direct violation, lands one level higher than the shape it concerns. Second, the `ex:ParttimeShape` branch is not discarded by a separate rule; its two children happen to be exactly the leaves that duplicate `ex:FulltimeShape`'s, so once those are folded into the survivors' alternate chains, `ex:ParttimeShape` has no children left and the pruning rule above removes it, leaving only its unique `ex:contractHours` leaf. Figure 4.4 makes both mechanisms visible by placing the tree exactly as built from the report above the same tree after deduplication.

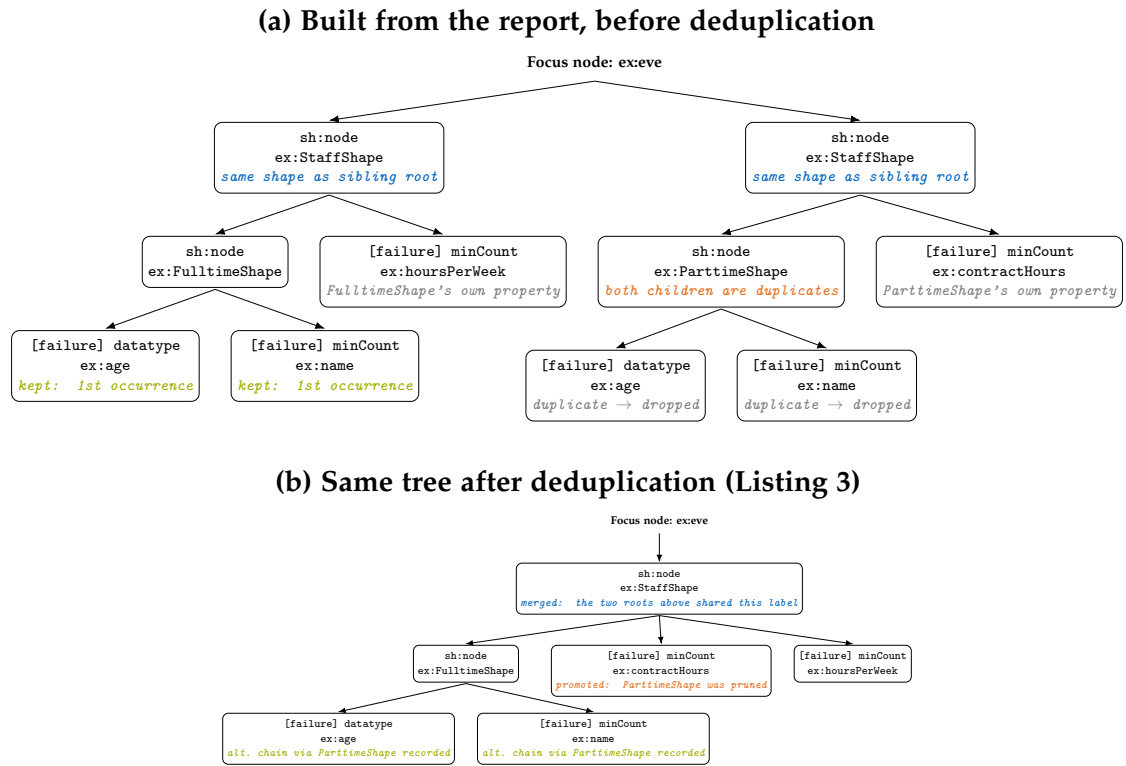


Figure 4.4.: Deduplication of the TC5 diamond, before and after. Two same-labelled root steps merge, two duplicate leaves are folded into alternate chains, and the resulting childless `ex:ParttimeShape` node is pruned.

The ordering of routes is preserved as far as possible, so that deduplication does not turn the explanation into an arbitrary reshuffling of the report: the first encountered chain remains the primary chain and later distinct chains become alternates. In TC7

there are no alternates; in TC5 the renderer displays the additional route as *also via*, while the summary can count both routes.

Deduplication therefore changes the unit presented to the user from "one validator result per route" to "one concrete failure with one or more routes." This distinction is reflected in Chapter 5, where leaf-failure counts refer to the deduplicated tree rather than to the raw number of `sh:ValidationResult` nodes.

4.8. Rendering and Output Modes

Readers need different views of one explanation without changing which failures count. All output modes consume the same filtered, deduplicated tree, summarized in Table 4.3. Filters on focus node, leaf path, constraint component and reference path are applied after construction, so they change what is shown rather than how reference chains are discovered.

Table 4.3.: The five presentations consume the same explanation tree.

Mode	Purpose	Main information preserved
Text	Human inspection in a terminal	hierarchy, paths, messages, chains and optional hints
JSON	Machine consumption	complete nested tree and alternate chains
CSV	Tabular analysis	one row per leaf failure with route information
Summary	Aggregate inspection	counts and rankings of paths, components and reference chains
HTML	Interactive exploration	complete tree, filtering, grouping and graph visualization

Terminal readers need to see each failure beneath its focus node and the complete route to it. The text view groups roots by focus node and shows reference steps as branches leading to concrete leaves. It appends the leaf's source shape to its stored chain only when three conditions hold:

- **The chain is already non-empty.** This is what distinguishes a referenced failure from a direct one: a direct violation has an empty chain, so nothing is appended and the chain stays empty.

- **The source shape is not a blank node.** This prevents an anonymous property shape from being printed.
- **That shape is not already present in the chain.** This prevents a shape from appearing twice in a row.

All output modes use the same namespace shortening, so an identifier does not change form when the representation changes.

Two small output labels are introduced here because they are renderer concerns. A **direct failure** is a leaf whose reference chain is empty; a **referenced failure** is a leaf reached through at least one `sh:node` step. The **chain depth** written by the summary and CSV output is the number of `sh:node` steps on that route; for a complete report, the reported depth is the maximum over its leaves.

Repair hints. A validator message may not state a safe local action, so hints are derived from leaf data and the relevant constraint parameter in the shapes graph. The rule covers only components for which such an action can be stated without additional reasoning. Table 4.4 shows the implemented cases.

Table 4.4.: Constraint components for which the renderer produces a repair hint. Other components remain fully reported but receive no generic hint.

Component	Generated local hint	Condition
minCount	Add at least one value on the failing path to the leaf focus node.	minimum of one, or unknown
minCount	Add the missing number of values, stating how many the node has and how many it needs.	minimum above one
maxCount	Remove values on the failing path until at most the declared maximum remain.	always
datatype	Replace the reported value with one of the required datatype, named where known.	value present
pattern	Change the reported value so that it matches the required pattern, quoted where known.	value present
class	Make the reported value an instance of the required class, or replace it.	property shape, value present
class	Give the leaf focus node a value on the path that is an instance of the required class.	property shape, no value
class	Make the leaf focus node an instance of the required class.	node shape
in	Replace the reported value with one of the allowed values, listed where known.	value present

A repair hint must identify the node to which the failed constraint applies. This may differ from both the original root focus node and the leaf’s focus node.

First, a property-level `sh:node` reference changes the focus from the original node to the referenced value, as described in Section 4.5. In TC7, `ex:CompanyShape` is checked on `ex:acme`, which is reached from `ex:alice` through `ex:worksFor`. The hint in Appendix B therefore tells the reader to add `ex:legalName` to `ex:acme`, not to `ex:alice`.

Second, the placement of the concrete constraint determines the hint target. A value

constraint such as `sh:class` on a property shape applies to each value reached through its `sh:path`, so the hint names that value. The same constraint on a node shape applies directly to the focus node. Always naming the focus node would therefore point to the wrong node for property-shape constraints.

Two things limit what a hint may claim. It states a requirement only where the shape that produced the result can be identified, which an external report does not always allow: Apache Jena and TopBraid report an anonymous property shape as a blank node that matches nothing in the shapes graph, and the reported path may then reach several property shapes. The parameter is taken only where all of them agree on it, so a hint never invents a requirement; otherwise the wording that states no parameter is used. The hints are also presentation aids rather than repairs in the formal sense: the tool does not claim minimality or that applying one suggested edit alone restores global conformance.

A complex property path is a blank-node structure whose label `rdflib` mints afresh on every parse. Printing it would name nothing and would differ between runs, so hints for such a path describe it instead of naming it.

Whether a hint is shown also depends on the output mode. The `--hints` flag gates the text tree only, as the TC7 text example in Appendix B shows. The JSON, HTML and CSV modes carry a repair hint on every leaf that yields one without being asked, because a consumer of those formats can ignore a field it does not want but cannot recover one that was never written.

Structured output. JSON and HTML consume the same serialized tree of dictionaries, so they share the meaning of each reference and leaf. An abbreviated TC7 JSON example is in Appendix B.

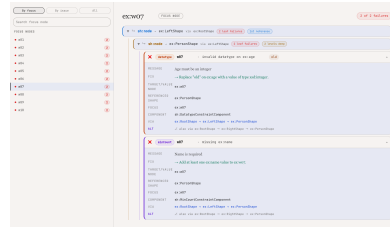
CSV intentionally flattens the hierarchy to one row per distinct leaf failure, retaining the reference chain and alternate chains so that the provenance lost by flattening can still be reconstructed during tabular analysis; Appendix C lists the ten written columns. The summary takes the opposite approach: it aggregates the same leaves into counts of affected focus nodes, direct versus referenced failures, failing paths, constraint components, reference paths and full leaf chains. Alternate chains participate in the chain statistics, so a diamond remains visible in aggregate analysis even though it is represented by one distinct leaf.

The HTML renderer uses the same serialized tree plus a separate shape catalog built from the shapes graph. The catalog is not needed to explain failures; it exists so that the graph view can optionally show constraints on the same shapes that *did not* fail. This distinction keeps the validation-derived explanation separate from schema context added for exploration. Section 4.9 describes that interface in detail.

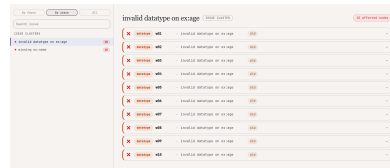
4.9. The Interactive HTML Report

The largest evaluation combination in Chapter 5 (Table 5.9) produces tens of thousands of leaf failures across tens of thousands of focus nodes, making a linear rendering difficult to inspect. The HTML mode therefore lets readers navigate, filter, group and graph the same explanation tree. It embeds the tree and a separate shape catalog in a self-contained page; Section 5.4 discusses the resulting output-size cost.

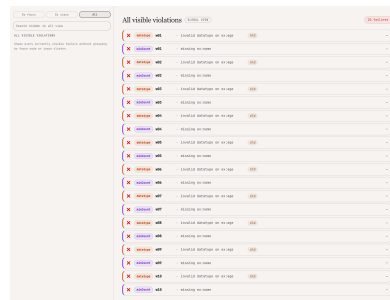
Sidebar navigation. The sidebar groups the same failures *By focus node*, *By issue* or in an ungrouped *All* list. The focus list initially shows twenty nodes, while search reaches entries beyond that preview. Figure 4.5 uses TC52 to show the three views when failures recur across nodes.



(a) **By focus.** Select a node to see its failures and alternate routes.



(b) **By issue.** Select a problem to see its affected nodes.



(c) **All.** Show every visible failure without grouping.

Figure 4.5.: Sidebar navigation on TC52: (a) by focus node, (b) by issue and (c) all visible failures without grouping.

4. Our Approach

Analytical view. The default expandable tree shows reference steps and leaf failures with their chains, messages and hints. Figure 4.6 shows TC7: a reference from `ex:alice` through `ex:worksFor` leads to the missing `ex:legalName` on `ex:acme`. The interface therefore shows the shift from root focus node to failing value.

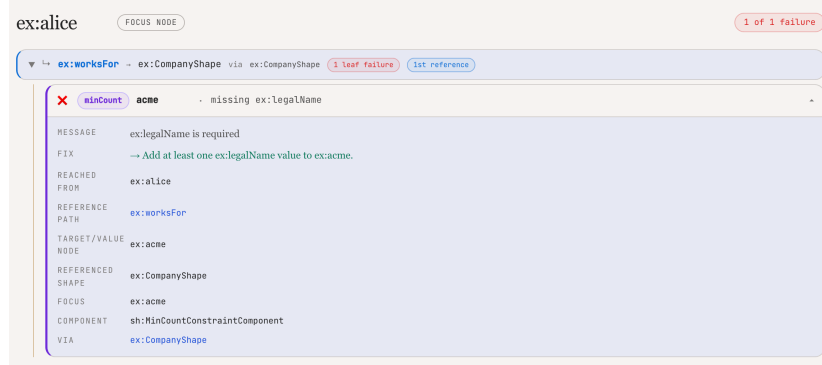


Figure 4.6.: Property-level `sh:node` in the analytical view, combining the reference-step and leaf-failure node blocks and showing that the concrete failure is attributed to the value node reached through the reference path.

Graph view. This view draws one focus node as the root, reference steps as intermediate nodes and failures as leaves. It is available in *By focus* mode, where one node anchors the layout. Figure 4.7 uses TC53 to show how reference depth remains visible along a longer chain.

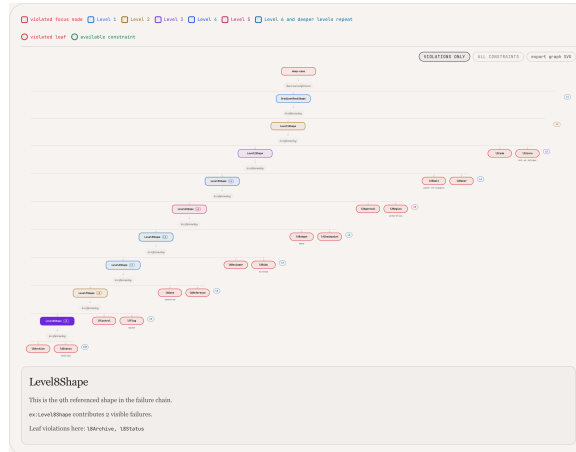


Figure 4.7.: Graph view of a nested reference chain, with reference depth encoded visually along the chain.

4. Our Approach

The graph can display either *Violations only* or *All constraints*. The first mode shows the explanation tree; the second adds declared constraints for which no violation was reported. Those constraints provide schema context and are not additional validation results. Figure 4.8 shows both modes.

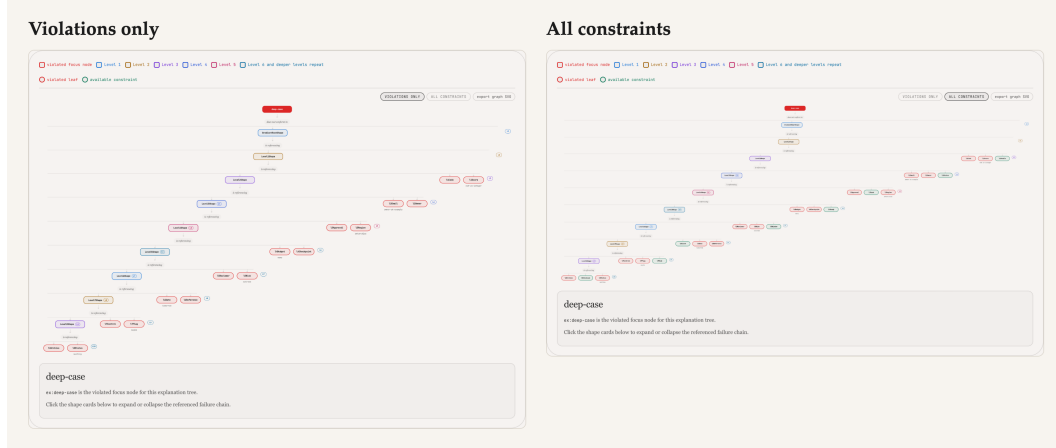


Figure 4.8.: The same graph in *Violations only* and *All constraints* modes. The latter supplements the explanation with non-violating constraints from the shapes graph: violated nodes are shown in red, non-violating constraint nodes in green.

Live filtering and summary. Browser-side filters narrow the report by constraint component, leaf path, reference path, source shape and failure kind (direct or referenced), updating the sidebar, headline counters and a root-cause summary together rather than merely hiding tree rows. In this output, a root cause is the leaf-level SHACL Core violation together with the route through the referencing constraints that led to it; the term denotes localization, not proof of a unique data edit. They compose with command-line filters: the CLI determines what enters the HTML artifact, while the browser determines which part of that artifact is currently visible.

4.10. Summary

This chapter presented the reconstruction pipeline that turns SHACL validation results into explanation trees. The implementation first preserves the report’s hierarchy by selecting only direct `sh:result` roots, then recursively separates referencing steps from concrete Core failures while recording the chain that connects them. Branch-local cycle detection guarantees termination without suppressing legitimate sibling routes. Where a report omits `sh:detail`, targeted re-validation reconstructs the missing local evidence without re-running the complete schema; where several routes converge on one failure, deduplication reports that failure once while preserving its alternate chains. Text, JSON, CSV, summary and HTML share the resulting tree. Chapter 5 evaluates these mechanisms on controlled fixtures and LUBM-scale data.

5. Experiments

The evaluation uses two complementary settings. First, 53 small fixtures support field-level checks of the generated explanations and cover the referencing patterns within scope. Second, six combinations of LUBM data graphs and shape schemas evaluate behavior on graphs containing millions of triples. The LUBM experiments measure performance and recurring failure groups. Independently enumerating every expected leaf failure and reference chain is impractical at this scale, so these experiments do not use a complete ground-truth explanation tree.

5.1. Experimental Setup

5.1.1. Functional Evaluation

The 53 Turtle fixtures under `all_test_cases/` cover nested and property-level `sh:` node, diamonds, cycles, direct failures, external reports and surrounding metadata. Appendix D lists all ten categories and their component coverage; Appendix E identifies the oracle for every fixture. Only `minCount`, `datatype`, `pattern` and `in` were tested as leaves reached through `sh:node`.

The suite comprises 52 automated tests, including end-to-end command-line checks, focused fallback and deduplication tests, full JSON comparisons and one browser-script test. Tests use subtests to exercise multiple fixtures. For 28 fixtures, the tests explicitly state the expected output fields. For the other 25, the generated explanation is compared with output previously stored from the implementation. These stored comparisons reveal unintended changes, but they cannot independently establish correctness because the baseline came from the same implementation. A separate smoke-test sweep verifies only that each command finishes successfully. Appendix E identifies the type of evidence available for each fixture.

5.1.2. Performance Evaluation

Two supplied LUBM-derived data graphs [GPH05], containing 1,001,716 and 4,258,329 triples, were each validated against three increasingly broad supplied shape schemas. Section 5.2 gives their sizes and reach. Schema2 and schema3 contain defects identified

by `tools/audit_shacl_schema.py`; results from the supplied schemas remain the primary results, while corrected copies test how much those defects contribute.

Each of the six combinations was run once for HTML output and once for summary output with timing enabled. Data were parsed from Turtle on every run, so parse times include loading the original files rather than a pre-parsed cache. Per-run measurements are recorded in `lubm_evaluation_results.md`.

5.1.3. Implementation Details

The implementation uses CPython 3.10.11, pySHACL 0.31.0 [SC26] and rdflib 7.6.0. Runs used macOS 15.6 on an Apple M4 machine with 10 cores and 24 GB of memory; the tool requested no parallel execution. Validation disabled inference and early exit, and passed the report to the explainer in memory without serializing it.

5.1.4. Metrics

Timing. *Parse time* is measured separately for the data graph, shapes graph and any supplied external report; the *LUBM timing table* includes only data-graph parsing because the shapes graphs are small and no external report is used. *Validate time* covers the top-level pySHACL call and its in-memory report. *Build-explanation-tree time* covers root extraction, recursive expansion, fallback re-validation and deduplication. *Render time* converts the finished tree to the selected output; filtering, limiting and CSV export are outside this stage. The *instrumented pipeline total* sums those stages; externally measured *wall-clock time* also includes startup, output writing and other uninstrumented work. When an external report is supplied, top-level validation is absent; any fallback re-validation still counts toward *tree-building time*.

Counts. Top-level pySHACL results are linked directly by `sh:result`; all-results also includes nested `sh:detail` results. Leaf-level results exclude `sh:NodeConstraintComponent` reference steps. Tool leaf failures are deduplicated by focus node, path, component and value, so this count should be compared with leaf-level results, not with all-results. *Affected focus nodes* counts roots of the surviving explanation forest, not every failing node within it; deduplication can remove a root. *Reference depth* counts `sh:` node steps to the deepest leaf.

Inputs and output. Schema reach is the number of distinct nodes selected by a schema's targets before validation; here all targets are `sh:targetClass`. HTML report size is the generated file size on disk. These measures distinguish the breadth of validation from the amount and cost of explanation.

5.1.5. Protocol and Reproducibility

The commands below are run from the root of the repository at <https://github.com/mariyakez/Root-Cause-Localization-for-Referencing-Constraints-in-SHACL>, which holds the implementation, the fixtures, the shape schemas and the generated reports.

The functional evaluation can be rerun from the repository root using the committed fixtures and assertions, without LUBM data:

```
python3 -m unittest discover -s tests -v
```

Every fixture-level figure in Section 5.3 was obtained by invoking the command-line interface on that fixture when preparing this chapter.

The three schemas, generated reports and per-combination measurements are committed. To rerun the six primary performance combinations, supply the original two LUBM graphs in Table 5.1, then run these commands from the repository root for each graph and schema in Table 5.2. Replace the placeholders with their paths and short names (for example, `skg1` and `schema1`). The corrected-schema check substitutes the corresponding file under `lubm_schemas/corrected/` in the summary command.

```
python3 -m shacl_explainer.cli <data> <shapes> --format html --hints --timing \
--output html_outputs/lubm_<dataset>_<schema>_report.html
python3 -m shacl_explainer.cli <data> <shapes> --summary --timing
```

The LUBM graphs (171 MB and 732 MB) are not committed. The repository permits inspection of recorded outputs, but exact result counts require the same supplied graphs.

5.2. Dataset and Shape Characteristics

Table 5.1 gives the two data graphs used in the performance evaluation. The larger graph holds roughly 4.25 times as many triples as the smaller one and is roughly 4.28 times its size on disk, so the two scale together and the pair can be read as one dataset at two sizes rather than as two unrelated inputs.

Table 5.1.: The two LUBM-derived data graphs used in the performance evaluation. Throughout this chapter they are abbreviated `skg1` and `mkg1`.

Data graph	Size	Data triples
lubm_skg_1.ttl	171.0 MB	1,001,716
lubm_mkg_1.ttl	732.0 MB	4,258,329

Table 5.2 gives the three shape schemas. Each is roughly double the size of its predecessor in shape triples, and the growth is not uniform in kind: the additional triples introduce new node shapes over new classes of the LUBM vocabulary, and they introduce new `sh:node` referencing constraints pointing at those shapes. The counts are the ones the tool reports, taken after validation has run. They exceed the counts obtained by parsing each file on its own, which are 52, 108 and 339, because pySHACL adds two fixed RDFS and OWL axiom triples to the shapes graph in place while validating. The post-validation counts are reported for consistency with the rest of the tool’s output, noting the difference here so neither figure is mistaken for an error.

Table 5.2.: The three SHACL shape schemas, in increasing order of size. Each data graph in Table 5.1 is validated against each of them, giving the six combinations reported in Section 5.3.

Schema	Shape triples
lubm_schemas/schema1.ttl	54
lubm_schemas/schema2.ttl	110
lubm_schemas/schema3.ttl	341

The distinction that matters for the results is not the schema sizes on their own but which parts of the data population they reach. `schema1` constrains degree-granting relationships, reaching the graph through `ub:undergraduateDegreeFrom`, `ub:mastersDegreeFrom` and `ub:doctoralDegreeFrom` into `ub:UniversityShape`. `schema2` adds course constraints reached through `ub:takesCourse` into `ub:CourseShape`, along with shapes for publications, research groups and undergraduate students. `schema3` adds `ub:GraduateCourseShape` and reaches it through both `ub:takesCourse` and `ub:teachingAssistantOf`, and it adds node shapes for lecturers, graduate students, teaching assistants, research assistants and the three professor ranks. The consequence is that the three schemas differ far more in the number of nodes they constrain than their triple counts suggest, and that difference is measurable without running a validation. Table 5.3 counts the focus nodes each schema’s `sh:targetClass` declarations select in each graph. Between `schema1` and `schema3` the shape triples grow by 6.3 times, while the nodes selected grow by 67.7 times on `skg1` and 97.8 times on `mkg1`. Against the 163,701 and 693,909 distinct typed nodes the two graphs contain, `schema1` selects 1.5 and 1.0 percent of them, `schema2` about 81 percent of each, and `schema3` all but a fraction of a percent. Section 5.3.3 uses these counts to separate the effect of schema reach from that of data size.

Table 5.3.: Schema reach: the distinct focus nodes each schema selects in each data graph, counted before validation. Every target in the three schemas is an `sh:targetClass`, and neither data graph carries an `rdfs:subClassOf` triple, so each figure is the number of distinct nodes typed with one of the targeted classes.

Data graph	schema1	schema2	schema3
skg1 (1,001,716 triples)	2,416	132,554	163,552
mkg1 (4,258,329 triples)	7,086	560,557	693,269

5.2.1. Schema Defects and the Corrected Variants

The schemas were checked against the data with `tools/audit_shacl_schema.py`, which reports three kinds of defect: a constraint path that occurs on no triple of the data, so that every targeted node fails a `sh:minCount` on it; an `sh:node` value that is not a shape of the schema, so that it carries no constraints and every value satisfies it; and a term of SHACL’s validation-result vocabulary used as a constraint parameter, which validators ignore. Table 5.4 lists what it found. `schema1` is clean; the other two are not, and the effects are large enough to decide what the evaluation measures.

Table 5.4.: Defects reported by the schema audit, with the effect each has on validation. `schema1` produced no findings.

Schema	Finding	Effect
schema2	<code>sh:path ub:type</code> on <code>ub:CourseShape</code> , a predicate absent from LUBM, where types are given with <code>rdf:type</code>	every course fails its <code>sh:minCount</code> by construction
schema2	<code>sh:value ub:Course</code> beside it, a result term and not a constraint	ignored by validators; the constraint is the <code>sh:minCount</code> alone
schema2	<code>sh:node ub:Course</code> on <code>ub:FullProfessorShape</code> , a class rather than a shape	carries no constraints, so every value satisfies it
schema3	the same three, with <code>ub:type</code> on <code>ub:CourseShape</code> and <code>ub:GraduateCourseShape</code> and eight class-valued <code>sh:node</code> references	as above
schema3	<code>sh:manCount 2</code> on <code>ub:FullProfessorShape</code> and <code>ub:LecturerShape</code> , a misspelling of <code>sh:maxCount</code>	ignored; the upper bound survives only because a sibling property shape on the same path spells <code>sh:maxCount</code> correctly

The corrected copies under `lubm_schemas/corrected/` repair exactly these findings, which Appendix F lists in full: the `ub:type` property shapes are removed, the class-valued `sh:node` references are pointed at the corresponding shapes, and `sh:manCount` becomes `sh:maxCount`. The corrections touch five triples in `schema2` and eighteen in `schema3`, leaving each file four and eight triples shorter, and the audit reports nothing on the result. One constraint that might look like a defect is deliberately kept: `ub:UniversityShape` requires an `ub:name`, and the universities that fail it are the ones the LUBM generator names as degree-granting institutions without ever describing them. That is a property of the data, not of the schema, and removing the constraint would hide it.

5.3. Results and Discussion

This section is organized around the five research questions introduced in Chapter 1, each answered in its own subsection and closing with a plainly stated answer. The first two (RQ1, RQ2) are answered on the fixture suite, where the expected output is known and correctness can be checked exactly. The last three (RQ3-RQ5) are answered on the LUBM runs, where it cannot. No figure from one group is carried into the other.

5.3.1. RQ1: Functional Coverage of Referencing Patterns

Observation. RQ1 is evaluated on the 53 fixtures described in Section 5.1.1. Each fixture is a small data graph and shapes graph containing a deliberate, known flaw. Together they cover the supported `sh:node` patterns: single and multiple hidden leaf failures, nested chains, several branches from one focus node, diamond references, property-level `sh:node`, self-recursion and mutual recursion, repeated structures, mixed direct and referenced failures, and an artificial eight-level chain in TC53. Fifteen further fixtures contain only direct constraints and serve as negative controls: they check that the explainer does not invent reference structure where none exists. The rest cover property paths, severities and message metadata. Table 5.5 gives the outcome for the fixtures that exercise referencing constraints, obtained by running the command-line interface on each in summary mode.

What counts as correct. The expected result is encoded in the 52 automated tests, not inferred from reading the generated reports. Several tests iterate over fixture groups with `subTest`, so a passing suite reaches all 53 fixtures although the runner reports 52 tests; the run completes with `Ran 52 tests and 0K`. Two kinds of oracle are used, and the strength of the claim differs between them. Appendix E records which applies to each fixture.

Fields (28 fixtures). The test names the strings that must appear in the output: the constraint component, the path, the value, the repair phrasing, or the classification as direct or referenced. These expectations were written independently of the output, so a passing test states that the named content is right. Nothing the test does not name is checked.

Stored JSON (25 fixtures). The complete explanation is compared against a saved document, every field of every node. Coverage is total, but the document was generated by the implementation. It therefore fixes current behavior as a regression baseline: it detects silent change, not an error already present when it was written.

Every ordinary fixture outside the external-report cases carries one further check, as do the directly validated fixtures in `fallback_edges` and `shared_path`. Each is also built a second way, by discarding the validator's `sh:detail` triples and recovering the nested results through re-validation. The two trees must agree field for field and in the same order, with one exception: the comparison omits the referenced-shape field for the five diamond fixtures TC4, TC5, TC6, TC26 and TC52 because that attribution cannot be reconstructed without `sh:detail` (Section 5.4). Both derivations share the tree building, deduplication and rendering beneath them, so an error common to both would escape. What the agreement does establish, and what no stored snapshot can, is that every compared field is independent of which source supplied the nested results.

Table 5.5.: Outcomes on the referencing fixtures. Depth is the number of `sh:node` steps traversed to the deepest leaf, as printed by the summary renderer. TC49 conforms, so no explanation tree is produced.

Case	Referencing pattern	Depth	Leaf failures (via <code>sh:node</code> / direct)	pySHACL results (all / top-level / leaf-level)
TC1	Single hidden leaf failure	1	1 (1 / 0)	2 / 1 / 1
TC2	Multiple hidden leaf failures	1	3 (3 / 0)	4 / 1 / 3
TC3	Two-level reference chain	2	1 (1 / 0)	3 / 1 / 1
TC4	Two <code>sh:node</code> branches, one focus node	1	3 (3 / 0)	5 / 2 / 3
TC5	Diamond reference	2	4 (4 / 0)	10 / 2 / 6
TC6	Nested organization hierarchy	2	12 (12 / 0)	23 / 6 / 14
TC7	Property-level <code>sh:node</code>	1	1 (1 / 0)	2 / 1 / 1
TC8	Cycle across property paths	1	1 (1 / 0)	26 / 3 / 9
TC46	Self-recursive shape	1	1 (1 / 0)	5 / 2 / 3
TC47	Two-shape cycle	2	2 (2 / 0)	5 / 2 / 3
TC48	Cycle with a leaf failure	1	1 (1 / 0)	2 / 1 / 1
TC49	Cycle without a leaf failure	n/a	reported as valid	n/a
TC52	Repeated diamond, ten focus nodes	2	20 (20 / 0)	80 / 20 / 40
TC53	Eight nested <code>sh:node</code> levels	8	16 (16 / 0)	24 / 1 / 16

Boundary cases. Four fixture families fix the limits of the claim. TC7 covers property-level `sh:node`, where the failure belongs to the referenced value node rather than the outer focus node; its fields are stated in advance and checked again against the real Jena and TopBraid reports. TC5 and TC52 cover diamonds, and verify that duplicate leaves collapse without discarding the alternate chains by which they were reached. TC46 to TC49 cover self-recursive and mutually recursive shapes and establish that traversal terminates. TC49 is the interesting one: a cycle containing no concrete violation, which pySHACL reports as conforming. The specification leaves validation with recursive shapes to the implementation [KK17, Section 3.4.3], so this outcome is pySHACL's choice, and the tool is tested on reproducing it rather than on deciding it. TC53 covers eight nested levels and establishes that depth alone does not prevent the leaf failures from being recovered. Of the four, TC5 and TC7 rest on stated fields; the cycle and depth cases rest on stored documents.

What the counts show about deduplication. Only the leaf-level column of Table 5.5 measures redundancy. The all-results count also includes the intermediate `sh:node` results, which record how a failure was reached rather than repeating it, so a ratio

taken against it reflects nesting as much as duplication: TC3 contains no duplicate failure at all and still gives a ratio of 3, and the eight levels of TC53 give 1.5 over sixteen distinct leaves. Against the leaf-level count the picture is unambiguous. Deduplication collapses 9 results to 1 in TC8, 40 to 20 in TC52, 6 to 4 in TC5, 3 to 1 in TC46 and 3 to 2 in TC47. The ratio is exactly 1 in TC1, TC2, TC3, TC4, TC7, TC48 and TC53, where the report holds no duplicate leaf to remove. The compression therefore acts on repeated leaf-level evidence and leaves the rest of the report untouched, which the two columns show directly rather than by inference.

Limits of the oracle. Two limits are visible in the stored documents. A complex property path reaches the output as the label of a blank node rather than as a readable path expression, so TC30 to TC33 establish that such a path survives as an opaque identifier, not that it is rendered intelligibly. For the external-report fixtures TC41 to TC45 the test asserts only that a summary is produced and that the expected path appears in it. Where a shape declares two `sh:message` values, TC38 and TC39 pin both in the stored document. The suite therefore checks positive reconstruction, the absence of reference structure where none should appear, and the stability of the rest, but it does not claim that every report feature is presented well. Correctness for arbitrary combinations of supported features at large scale is treated in Section 5.4.

Answer to RQ1. Yes, for the referencing patterns the fixture suite represents, and at the per-fixture granularity of Appendix E rather than uniformly across all 53. All 52 tests pass, so every assertion the suite makes holds. Every pattern the tool claims to support is covered by at least one fixture, and failed constraint, path, value, chain and repair hint are each asserted somewhere, all five together on the stored-JSON fixtures. The strength of that coverage is uneven: property-level attribution and alternate-route preservation in a diamond are stated in advance, whereas termination on cycles and eight-level traversal rest on documents captured from the implementation and so establish stability rather than independently derived correctness.

5.3.2. RQ2: Reconstruction from External Reports

Observation. RQ2 uses the external-report fixtures TC41 to TC45 together with the focused fallback tests in `test_fallback.py`. Each instantiates one row of Table B.1, which classifies what such a report withholds and what the fallback recovers in each case, and which is not repeated here: TC41 is the control, a pySHACL-style report that already contains `sh:detail`, and TC42, TC43, TC44 and TC45 take the first, second, third and fourth rows in turn. TC43 and TC44 assume that a property-level result identifies either the enclosing node shape or the referenced shape through `sh:sourceShape`. However, the real validators examined below do not follow either pattern. Instead, they identify the property shape itself. Passing `--report` suppresses the initial

5. Experiments

top-level validation, but the fallback may still invoke pySHACL on a minimal shapes graph to recover omitted nested results. Table 5.6 gives what each produced when run with `--report` and `--hints`.

Table 5.6.: Reconstruction from external validation reports. TC41 is the control in which `sh:detail` is present.

Case	Report style	sh:detail?	Reference node recovered	Leaf failure recovered	Repair hint
TC41	pySHACL, with detail	yes	sh:node ex: PersonShape	minCount ex :name	add ex:name to ex:alice
TC42	Jena, no detail	no	sh:node ex: PersonShape	minCount ex :name	add ex:name to ex:alice
TC43	Synthetic, enclosing shape	no	sh:node ex: CompanyShape, path=ex:worksFor	minCount ex :legalName	add ex:legalName to ex:acme
TC44	Synthetic, referenced shape	no	sh:node ex: CompanyShape, path=ex:worksFor	minCount ex :legalName	add ex:legalName to ex:acme
TC45	No sh:sourceShape	no	none	minCount ex :name	add ex:name to ex:alice

The table reports what each fixture produced, which is more than the tests assert for it: for TC41 to TC45 the assertion is only that a summary is produced and that the expected path appears in it. The referenced shape, the concrete leaf constraint, the value node, the repair hint and, for a diamond, the preservation of alternate reference chains are checked elsewhere, by the focused fallback tests of `test_fallback.py` and by the comparison against the real validators' reports below. Read with that qualification, TC41 and TC42 produce the same explanation structure despite only the former carrying `sh:detail`. The two synthetic property-level cases likewise recover the same leaf failure and repair information from reports that omit the nested results. Two of the direct fallback tests check the mechanism independently of any external validator. They generate pySHACL reports in-process, delete every `sh:detail` triple and then call `build_explanation_tree`. On TC2 the fallback recovers all three expected concrete failures; on TC5 it reconstructs both sides of the diamond and leaves deduplication with the same alternate-route information as the original detailed report.

Validation against real validators. The fixtures above are hand-written imitations,

so they establish only that the tool handles the report structures they encode. To test the premise itself, every referencing fixture of Table 5.5 was validated with two real validators, Apache Jena SHACL 6.2.0 [Apa26], invoked as `shacl validate --shapes F --data F`, and TopBraid SHACL 1.5.0 [Top26], invoked as `shaclvalidate.sh -datafile F -shapesfile F`, and each resulting report was fed back through `--report`. The two share Jena’s RDF library but not its SHACL engine: the TopBraid distribution bundles `jena-core` and not `jena-shacl`, so validation and report construction are implemented separately in each. Explanations were compared in their JSON form, reduced to the leaf failures, each identified by focus node, path, constraint component, value and repair hint, together with the reference chains attached to each, so that the order in which roots and siblings are printed (Section 5.4) does not count as a difference.

The premise holds for both. Neither validator emitted a `sh:detail` triple on any of the fourteen fixtures, and TopBraid’s command-line tool offers no option to request one. Both returned the same conformance verdict as pySHACL throughout, including TC49 remaining valid; on this recursive fixture that is an agreement between three implementations’ choices, not a behavior the specification prescribes. On all thirteen non-conforming fixtures, reconstruction from either validator’s report recovered exactly the leaf failures of the pySHACL-native explanation. The reference chains attached to them agreed as well on every fixture for TopBraid, and on every fixture but TC46 for Jena. This is stronger evidence for RQ2 than the hand-written fixtures give, because the input was produced by independent implementations rather than written to match the expectation being tested.

The same comparison was then run on the remaining thirty-four fixtures that are not themselves hand-written reports: the direct, mixed, multi-focus, property-path, severity, message-metadata and scale cases. For both validators it found no difference at all; every leaf failure, chain and repair hint equalled pySHACL’s native explanation, with leaf messages set aside because each validator words its own and blank-node path labels normalized as in Section 5.4. Since most of these fixtures contain no referencing constraint, this establishes that the report features they exercise pass through a real validator’s report unchanged, rather than anything further about reconstruction. The sweep also covers the conforming case, which the hand-written report fixtures do not contain: TopBraid’s report for TC49 states that the data conforms, and the tool reports it as valid rather than producing an empty explanation. That report is committed and asserted by its own test.

Where the validators differ. The reference chains differ on one recursive fixture, TC46, and only for Jena: Jena reports only the direct `minCount` result and no `sh:n` node result for the self-reference, so the reconstructed leaf carries no chain, whereas TopBraid’s report reproduces pySHACL’s. The top-level result counts show the same

divide: TopBraid’s match pySHACL’s on all fourteen fixtures, whereas Jena’s differ on TC8 and TC46. Both differences fall on a recursive shape, which is consistent with the absence of a prescribed semantics for recursion rather than with a defect in either tool, and the failures a user has to repair are the same in every case.

A corrected premise. The exercise also corrected a mistaken premise in the hand-written fixtures. For a property-level `sh:node` result, TC43 names the enclosing node shape in `sh:sourceShape` and TC44 names the referenced shape. Neither validator does either. Both name the node shape for a node-level result, as TC42 assumes, but for a property-level result both name the *property* shape. Where that shape is named, the report carries its name; in the usual idiom it is anonymous, so `sh:sourceShape` is a blank node that matches nothing in the shapes graph. Such a result therefore identifies its constraint only through the reported path, and that is the route the reconstruction of Section 4.6 takes: it locates the property shapes of the shapes graph that declare `sh:node` on that path and narrows them to the one the result came from where several match. Without that route nothing in these reports identifies the referenced shape, so a suite built only on the hand-written fixtures would leave the case untested. The real reports of both validators for TC7 are committed as `jena_real_tc7_property_no_detail.ttl` and `topbraid_real_tc7_property_no_detail.ttl` and asserted by the same test. TC43 and TC44 are retained under names that describe them as synthetic, as robustness cases for report structures that neither validator produces.

What the re-validation copy must contain. Running the reconstruction over the LUBM reports shows what the targeted shapes graph of Section 4.6 has to leave out, and why the fixture suite alone cannot show it. A referenced shape often selects focus nodes of its own: `ub:CourseShape` carries `sh:targetClass ub:Course`. A copy that kept that declaration would validate all 8,126 courses of `skg1` whenever one course is re-validated, and deduplication would file the failures of unrelated nodes under whichever step reached them first, so the copy drops every target declaration, explicit or implicit. Among the fixtures only the recursive ones have a referenced shape with a target at all, and there it selects the very node being re-validated, which is why the LUBM reports expose the requirement and the fixtures do not. A fixture whose referenced shape targets its whole class, with real reports from both validators, asserts that a rebuilt step contains the failure of its own value only (`fallback_edges/targeted_referenced_shape.ttl`).

Comparing the rebuilt explanation with the one read from `sh:detail`, fixture by fixture, pins the second requirement on that copy: it must carry the whole structure a shape depends on, which is why it follows every blank node below a shape and every named shape it refers to, at any depth. A copy that stopped short would not merely lose detail but change the answer. An `sh:in` list without its members is empty, so every value fails it, which TC53 exercises; a shape referred to through a property-level `sh:node`

e inside the referenced shape, left uncopied, carries no constraints, so every value passes it, which `fallback_edges/nested_property_node.ttl` exercises from both validators' reports. With the copy complete, with its messages worded under the shapes graph's prefixes and with the children of a merged step ordered by content, removing `sh:detail` from pySHACL's own report leaves the explanation of every fixture unchanged byte for byte, with and without the re-validation cache of Section 4.6, which a test requires.

Limits of reconstruction. Three things are not recovered. Each is a limit of what the input carries rather than a failure of the method.

TC45 defines the first. Once `sh:sourceShape` is absent, the report no longer identifies the referenced shape against which the value node should be re-validated. The concrete leaf failure and its repair hint survive, because the report carries them directly, but the reference chain cannot be rebuilt. This is an information limit of the input rather than evidence that the fallback fails when the referenced shape is available.

The second applies to the hints alone and not to the failures themselves. A hint names what the shape requires, and that requirement is read from the shape the result identifies, or, where a report identifies only an anonymous property shape, from the reported path. Neither is available when the constraint sits on a complex property path, because the path is itself a blank node that no longer matches the shapes graph once the report has been written to a file. The hint then states the failure without the requirement, as it does for TC33, which is a weaker hint rather than a different failure.

The third does not appear in these measurements at all. Where a shape declares several `sh:node` references, which of them a step travelled is recorded only in `sh:detail` and is not recovered. Neither the fixtures nor the LUBM schemas exercise this, so it stands here as a known gap rather than an observed difference.

Cost of reconstruction at scale. The performance evaluation of Section 5.3 never exercises the fallback: pySHACL attaches `sh:detail` to its `sh:node` results, so those runs read the nested results instead of rebuilding them. Reconstruction is also the one part of the pipeline whose cost is not bounded by the size of the report, since it re-validates once per `sh:node` result. It was therefore measured separately, on the same two data graphs and three schemas: each combination was validated with pySHACL, every `sh:detail` triple was deleted from the report, and the whole explanation was built again through the fallback. Table 5.7 gives the result, in the same process rather than through the command line, so its figures are comparable with each other but not with the pipeline stages of Table 5.8.

Table 5.7.: Cost of reconstruction when `sh:detail` is removed, against the normal path that reads it. *Calls* counts the pySHACL re-validations. One run per combination.

Dataset $\times$ Schema	<code>sh:node</code> results	From <code>sh:detail</code> 1	Rebuilt, shared (calls)	Rebuilt, one call per result (calls)
<code>skg1 $\times$ schema1</code>	1,144	0.09 s	0.52 s (655)	0.84 s (1,144)
<code>skg1 $\times$ schema2</code>	179,677	12.99 s	22.59 s (8,781)	173.00 s (179,677)
<code>skg1 $\times$ schema3</code>	257,035	22.30 s	37.55 s (17,043)	248.38 s (257,035)
<code>mkg1 $\times$ schema1</code>	4,868	0.39 s	1.17 s (958)	4.57 s (4,868)
<code>mkg1 $\times$ schema2</code>	759,929	57.94 s	88.66 s (35,597)	642.48 s (759,929)
<code>mkg1 $\times$ schema3</code>	1,091,892	118.02 s	215.93 s (69,729)	931.05 s (1,091,892)

Three things follow. First, reconstruction reproduces at LUBM scale what it reproduces on the fixtures: on all six combinations the rebuilt explanation is identical to the one read from `sh:detail`, field for field and in the same order, with the same summary counts, for up to 87,396 leaf failures rebuilt from 1,091,892 `sh:node` results. This is the one check of the explanation at scale that does not need an independently derived expected tree, since the two paths must agree; Section 5.4 states what it does not establish. Second, the unshared cost is linear in the number of `sh:node` results, at 0.64 to 0.88 milliseconds per result on average and at most 1.08 milliseconds at the 95th percentile, which is 7.9 to 13.3 times the cost of reading `sh:detail` and, on the largest combination, about twice the validation it follows. Third, sharing re-validations across a build (Section 4.6) removes most of that: the same nodes are re-validated against the same shapes again and again, so the number of calls falls by 1.7 to 21.3 times, and the rebuilt explanation costs 1.5 to 3.0 times the `sh:detail` path on the five combinations larger than `skg1/schema1`.

Two qualifications. The runs are single runs on one machine, as in Section 5.3, and the measurement holds more in memory than the command line does, since it keeps the data graph, the report and two complete explanations at once; its peak was 14.0 GB on the `mkg1` combinations. The `sh:detail` figure for `mkg1  $\times$  schema3` is the one number visibly affected: at 118.02 seconds it is slower than the same build measured through the command line (85.502 seconds, Table 5.8), and its per-result cost, 0.108 milliseconds, is the highest of the six combinations against 0.072–0.087 milliseconds for the rest. The ratios for that row should therefore be read as conservative.

Interpretation. The tested external formats show that `sh:detail` is not required for reconstruction when the report still identifies the source shape or the failing path. Two real validators now support that conclusion across the whole referencing suite rather than one hand-written report per structure. It remains evidence about three implementations, pySHACL, Apache Jena and TopBraid, on these fixtures, and not about every validator-specific serialization or extension.

Answer to RQ2. Yes, provided the external report identifies the referenced shape or the path leading to it. Against real Apache Jena 6.2.0 and TopBraid SHACL 1.5.0 output, neither of which carries `sh:detail`, reconstruction recovered the leaf failures of the pySHACL-native explanation on all thirteen non-conforming referencing fixtures; the reference chains differed on one recursive fixture, and only for Jena. The three things reconstruction does not recover are set out above: the chain when `sh:sourceShape` is absent, the requirement in a hint when the constraint sits on a complex property path, and which of several `sh:node` references a step travelled. Rebuilding costs one re-validation per `sh:node` result, which is where the only unbounded cost of the pipeline sits; sharing re-validations across a build keeps it at 1.5 to 3.0 times the cost of reading `sh:detail`, and the rebuilt explanation is identical to the one `sh:detail` gives. This is evidence about three implementations, pySHACL, Apache Jena 6.2.0 and TopBraid SHACL 1.5.0, on these fixtures and the six LUBM combinations; it is not evidence about external reports in general, and the five hand-written report fixtures of the table above establish less on their own than the real reports do.

5.3.3. RQ3: Scaling with Data Size and Schema Reach

Observation. Table 5.8 gives the timing breakdown for all six combinations, in seconds, from the pipeline’s own instrumentation. The pipeline total is the sum of the timed stages, including the shapes-parse stage that is too small to appear separately.

Table 5.8.: Runtime breakdown per stage, in seconds, one run per combination.

Dataset × Schema	Parse data	Validate	Build tree	Render	Pipeline total
skg1 × schema1	19.188	0.829	0.095	0.045	20.16
skg1 × schema2	19.267	62.041	10.194	0.338	91.84
skg1 × schema3	19.158	92.290	17.751	0.645	129.84
mkg1 × schema1	84.199	3.225	0.291	0.068	87.78
mkg1 × schema2	83.941	302.849	50.090	1.309	438.19
mkg1 × schema3	83.167	471.000	85.502	2.783	642.45

Parsing as a noise floor. Parse time depends only on the data graph. It remains

around 19 seconds on skg1 and 83 to 84 seconds on mkg1 regardless of which schema follows: the three skg1 measurements differ by 0.1 seconds, the three mkg1 measurements by 1.0 seconds. Because parsing never sees the schema, that spread cannot be a schema effect, and it therefore gives a direct estimate of the run-to-run noise visible at this scale. Every difference reported below is one to two orders of magnitude larger than it.

The two scaling axes. Validation behaves quite differently on the two axes. On fixed data, moving from schema1 to schema3 increases validation from 0.829 to 92.290 seconds on skg1, a factor of 111, and from 3.225 to 471.000 seconds on mkg1, a factor of 146. On a fixed schema, increasing the data by 4.25 times increases validation by 3.89 times under schema1, 4.88 under schema2 and 5.10 under schema3. Quadrupling the data therefore roughly quadruples the cost, whereas exchanging the schema multiplies it by more than a hundred. Section 5.4 reports how far these factors move between runs.

Schema reach, counted independently in Table 5.3, moves with both figures. Across the schema step it rises by 67.7 times on skg1 and 97.8 times on mkg1, and quadrupling the data at a fixed schema raises it by 2.93 to 4.24 times. Table 5.9 gives the corresponding report and explanation sizes.

Table 5.9.: pySHACL result counts and explanation output counts per combination.

Dataset × Schema	pySHACL results (all / top-level)	Focus nodes affected	Leaf failures (via sh:n ode / direct)
skg1 × schema1	3,280 / 2,136	870	992 (655 / 337)
skg1 × schema2	369,453 / 189,776	6,306	10,099 (8,781 / 1,318)
skg1 × schema3	537,637 / 280,602	16,207	21,170 (17,043 / 4,127)
mkg1 × schema1	10,704 / 5,836	839	968 (958 / 10)
mkg1 × schema2	1,559,594 / 799,665	25,032	39,736 (35,597 / 4,139)
mkg1 × schema3	2,281,457 / 1,189,565	67,941	87,396 (69,729 / 17,667)

Cost per unit of work. The unit costs say where that difference comes from. Validation cost per target node stays between 0.34 and 0.68 milliseconds across all six combinations, and validation cost per pySHACL result between 0.167 and 0.306 milliseconds. Each varies by less than a factor of two, against a 111- to 146-fold spread in validation time and a 696-fold spread in the number of results. The tool is therefore not becoming less efficient under the larger schemas; it is being given far more to do. The result counts confirm that volume: from schema1 to schema3 all pySHACL results rise from 3,280 to 537,637 on skg1, a factor of 164, and from 10,704 to 2,281,457 on mkg1, a

factor of 213, whereas the data-size step at a fixed schema raises them by only 3.3 to 4.2 times, close to the 4.25 ratio in data triples. Build-tree time follows the same pattern and is taken up in Section 5.3.4.

More data, no more distinct failures. One combination separates data size from schema reach especially clearly, provided it is read carefully. Under schema1, the four-times-larger mkg1 graph produces 968 leaf failures, slightly fewer than the 992 on skg1, and 839 affected focus nodes, again slightly fewer than the 870 on skg1. The validation evidence behind those numbers is not flat: top-level pySHACL results rise from 2,136 to 5,836 and all results from 3,280 to 10,704, both close to the 4.25 ratio in data triples. The stable leaf count is therefore not an absence of additional failures but the effect of two bounds that the extra triples do not move. Schema1’s dominant failure is the missing `ub:name` on universities named as degree-granting institutions, and both graphs reach a similarly bounded set of university IRIs through those properties. Deduplication then identifies each such failure by focus node, path, component and value (Section 4.7), so the many additional students and departments in mkg1 that point at the same universities collapse onto the same leaves rather than adding new ones. More triples consequently need not imply proportionally more *distinct* failures, even where they do imply proportionally more validation results.

How much of this is the schema defects? The growth across schemas is driven by the number of results a schema causes, and two of the defects of Section 5.2.1 create results in bulk: every course fails `ub:type`, and every reference into a course shape reaches a node that fails. Repeating the summary run of each affected combination against the corrected schemas separates the two. Table 5.10 gives the comparison.

Table 5.10.: Supplied against corrected schemas, one summary run each, on the same machine and code. schema1 is unchanged by the corrections and is repeated for reference.

Dataset $\times$ Schema	Top-level results		Leaf failures		Validation (s)	
	supplied	corrected	supplied	corrected	supplied	corrected
skg1 $\times$ schema1	2,136	2,136	992	992	0.8	0.8
skg1 $\times$ schema2	189,776	3,117	10,099	1,973	62.0	25.3
skg1 $\times$ schema3	280,602	22,555	21,170	5,036	92.3	42.9
mkg1 $\times$ schema1	5,836	5,836	968	968	3.2	3.2
mkg1 $\times$ schema2	799,665	9,965	39,736	5,097	302.8	106.1
mkg1 $\times$ schema3	1,189,565	93,058	87,396	18,279	471.0	184.9

The defects account for most of the load but not for the effect. Removing them removes 92 to 99 percent of the top-level results and 76 to 87 percent of the leaf failures,

and validation falls to between a third and a half. What survives is the direction of the finding: on the corrected schemas, moving from schema1 to schema3 on fixed data still multiplies validation by 52 times on skg1 and 57 times on mkg1, against 111 and 146 times on the supplied schemas, while quadrupling the data still multiplies it by about four. Schema reach still dominates data size; the supplied schemas overstate by roughly a factor of two how steeply it does so.

Interpretation. On these inputs, validation time is governed more directly by the number of validation results produced than by schema triples or data triples in isolation. Section 5.2 provides the structural explanation: schema2 and schema3 add constraints through relations such as `ub:takesCourse` and `ub:teachingAssistantOf`, which reach far more nodes than schema1’s degree-granting relationships. Table 5.3 counts the focus nodes each schema selects before validation, so the class populations are measured rather than inferred. Reach and result volume are therefore two measurements of the same effect rather than one explaining the other, and reach is the one that can be obtained without running a validation.

Answer to RQ3. Schema reach dominates data size in this evaluation. Quadrupling the data increases validation by roughly 4–5 times, whereas moving from schema1 to schema3 on fixed data increases it by 111–146 times on the supplied schemas and, once their defects are repaired, by 52–57 times. The nodes the schemas select, counted before validation in Table 5.3, grow by 67.7 and 97.8 times across the same step, and the corresponding pySHACL result counts by similarly different factors, while validation cost stays within a factor of two per target node and per result alike. The practical scaling variable is therefore how many nodes a schema’s targets select in the graph, which can be counted from the schema and the data before any validation is run; the result volume follows it.

5.3.4. RQ4: Cost of the Explanation Layer

Observation. Table 5.11 isolates the two stages this thesis contributes, tree building and rendering, and expresses them against two baselines: the pySHACL validation they post-process and the pipeline total including parsing. The ratios are derived from Table 5.8.

Table 5.11.: Cost of the explanation layer against two baselines. Ratios are derived from the measurements in Table 5.8.

Dataset $\times$ Schema	Build tree (s)	Render (s)	Build / validate	Build / pipeline total	(Build + render) / pipeline total
skg1 $\times$ schema1	0.095	0.045	11.5%	0.5%	0.7%
skg1 $\times$ schema2	10.194	0.338	16.4%	11.1%	11.5%
skg1 $\times$ schema3	17.751	0.645	19.2%	13.7%	14.2%
mkg1 $\times$ schema1	0.291	0.068	9.0%	0.3%	0.4%
mkg1 $\times$ schema2	50.090	1.309	16.5%	11.4%	11.7%
mkg1 $\times$ schema3	85.502	2.783	18.2%	13.3%	13.7%

The two baselines. Against validation, building the explanation tree costs 9.0 to 19.2 percent across the six combinations. The largest relative value occurs on skg1 $\times$ schema3, where 17.751 seconds of tree construction follows 92.290 seconds of validation; on the largest absolute run, mkg1 $\times$ schema3, 85.502 seconds follows 471.000 seconds, or 18.2 percent. Tree construction therefore remains below one fifth of validation in every combination, with the least margin on skg1 $\times$ schema3 at 19.2 percent.

Against the full instrumented pipeline the share is smaller, 0.3 to 13.7 percent, and the schema1 rows show why the two baselines differ. Validation is so cheap there that Turtle parsing dominates the command, so tree building can be around ten percent of validation while contributing less than one percent of the whole pipeline; under schema2 and schema3, where validation dominates, the two ratios converge. Both are reported because they answer different questions. The build-to-validate ratio is the cleaner measure of the explanation algorithm’s overhead, whereas the build-to-pipeline ratio better captures the extra delay a user experiences once the unavoidable input-parsing cost is included.

Rendering is negligible on either baseline. It remains below 2.8 seconds in every combination, including 2.783 seconds on mkg1 $\times$ schema3, about 0.43 percent of that 642.45-second pipeline. The practical explanation-layer overhead is therefore almost entirely tree construction rather than serialization of the selected output.

Cost per result is nearly constant. Absolute build time grows from 0.095 to 85.502 seconds across the six combinations, which on its own could look like a second scalability problem alongside validation’s. Normalizing by report size dissolves it. Dividing build-tree time by the all-results count yields 0.0290, 0.0276, 0.0330, 0.0272, 0.0321 and 0.0375 milliseconds per pySHACL result in table order: while the report grows by a factor of 696, the normalized cost varies only from 0.0272 to 0.0375 milliseconds,

a factor of 1.38. Build time therefore grows roughly in proportion to the amount of validation evidence presented to the explainer rather than on its own terms. This does not establish an asymptotic complexity bound, but it is a more informative comparison than absolute time alone.

How much of this is the schema defects? The corrected schemas support the same reading from the other direction. With the by-construction failures gone, both validation and explanation shrink, and the explanation shrinks faster: tree construction falls from 10.19 to 0.42 seconds on skg1 under schema2 and from 85.50 to 4.97 seconds on mkg1 under schema3, which is 1.6 and 2.7 percent of the corresponding validation times rather than 16 and 18 percent. The layer is cheapest, relative to validation, exactly where the report is not inflated by a defect.

Interpretation. The near-constant per-result cost reads as evidence that tree construction is *effectively linear in report size on these inputs*. The wording is deliberately empirical rather than theoretical. Only six performance points were measured, all use the same LUBM vocabulary and the large schemas exercise breadth far more than depth. In particular, their reference chains contain at most two `sh:node` steps. The eight-level chain in TC53 establishes that deeper traversal works functionally, but its graph is too small to show whether the same per-result behavior persists for deep chains at scale. Likewise, the narrow normalized-cost range cannot rule out super-linear behavior outside the measured range. Within the experiment, however, there is no indication that recursive expansion or deduplication adds a rapidly increasing cost as the report grows.

Answer to RQ4. No, the explanation layer is not the bottleneck at the measured scales. Tree construction remains below one fifth of validation time and below 14 percent of the instrumented pipeline, while rendering is negligible. Its normalized cost stays nearly constant across almost three orders of magnitude in report size, with the important qualification that deep reference chains were not varied at scale.

5.3.5. RQ5: Consistency of Findings Across Scale

Observation. RQ5 asks whether the same diagnosis recurs when the data grows, which concerns the identity of the dominant patterns and the repair category they point to rather than equality of the raw failure counts between datasets. The same two root-cause patterns appear on both LUBM graph sizes. Table 5.12 shows the principal chain-attributed groups that support this comparison.

Table 5.12.: Largest leaf-failure groups by reference chain. Entries marked n/a correspond to shapes not present in that schema.

Dataset × Schema	ub:UniversityShape -> minCount ub:name	ub:CourseShape -> m inCount ub:type	ub:GraduateCourseSh ape -> minCount ub: type
skg1 × schema1	655	n/a	n/a
skg1 × schema2	655	8,126	n/a
skg1 × schema3	992	8,126	7,925
mkg1 × schema1	958	n/a	n/a
mkg1 × schema2	958	34,639	n/a
mkg1 × schema3	968	34,639	34,122

The two recurring patterns. The first is missing `ub:name` on universities reached through degree properties into `ub:UniversityShape`. Under `schema1` this is the largest chain-attributed group on both datasets, with 655 failures on `skg1` and 958 on `mkg1`. Looking across schemas, the absolute number of missing-name failures is stable within each dataset: 992 on `skg1` and 968 on `mkg1`. The table reports fewer than that under `schema1` and `schema2` because its column counts only the failures a reference chain reached; the remainder are reported as direct failures, for the reason given under the direct and referenced columns below.

The second pattern is missing `ub:type` on course-related nodes, and it is larger whenever the relevant shapes are present. Under `schema2`, `ub:CourseShape -> minCount ub:type` contributes 8,126 failures on `skg1` and 34,639 on `mkg1`. `Schema3` retains that pattern and adds the comparable `ub:GraduateCourseShape` cluster, with 7,925 failures on `skg1` and 34,122 on `mkg1`.

The lower-level reference-path breakdown supports the same conclusion rather than merely the aggregate clusters. Under `schema3`, the three largest paths on `skg1` are `ub:takesCourse -> ub:GraduateCourseShape` (5,739), `ub:teachingAssistantOf -> ub:CourseShape` (4,156) and `ub:takesCourse -> ub:CourseShape` (3,058). On `mkg1` the same three paths occur in the same order, at 24,511, 17,839 and 12,945 failures respectively. The counts increase substantially with the larger graph, but the identity and ordering of the principal paths do not change.

Reading the direct and referenced columns. What changes across schemas is not the number of these failures but their attribution: 337 of the 992 are reported as direct on `skg1` under `schema1` against 10 of 968 on `mkg1`, while under `schema3` all of them are reached through a reference chain. That split is a property of the presentation rather than of the data. The same missing `ub:name` can be reported by `pySHACL` both as a direct target of `ub:UniversityShape` and below a `sh:node` step from a degree property,

and the two reports share a deduplication key, so only the occurrence deduplication meets first survives and decides the classification. Because `build_explanation_tree` sorts top-level results so that `sh:node` results are expanded first, the referenced occurrence wins whenever one exists. The direct column therefore counts the failures that no reference chain reached in that run, and it shrinks as a schema's reference structure covers more of the same population, which is what happens from `schema1` to `schema3`. The difference between 337 and 10 is consequently not a difference between the two graphs.

What is stable and what is not. The university cluster does not remain the largest under every schema: it dominates `schema1`, but the course clusters overtake it under `schema2` and `schema3`. What is stable across scale is the identity and ordering of the dominant patterns for a given schema, not a single universal winner across all schemas. The scale change itself behaves consistently: the course-related groups grow by roughly the data-size factor from `skg1` to `mkg1`, whereas the university failures remain near one thousand on both graphs, which reinforces the RQ3 result that the population reached by a constraint matters more than graph size in the abstract. The repair category is stable for the same reason the counts are not, because the underlying leaf constraint does not change with scale: the university cluster points to adding the missing `ub:name`, while the course clusters point to supplying the required `ub:type`.

What the recurring patterns are. The explanation does more here than rank groups: it says what each group reduces to, and neither of the two largest is a data error. The `ub:type` group exists because no such predicate occurs in LUBM at all, so every course fails the constraint by construction; the group sizes are exactly the course populations, 8,126 on `skg1` and 34,639 on `mkg1` under `schema2`, and both course classes together under `schema3`. The `ub:name` group is of a different kind: the constraint is satisfiable, but the LUBM generator names 1,000 universities as degree-granting institutions while describing only 8 of them on `skg1` and 32 on `mkg1`, so the 992 and 968 that remain carry no name. One is a defect of the schema and the other a gap in the generated data, and the tool located both by reducing tens of thousands of results to a handful of chains.

How much of this is the schema defects? Repeating the runs against the corrected schemas confirms the reading. The `ub:type` group disappears entirely, and the largest remaining group is `ub:teacherOf` `maxCount`, which is a direct failure rather than one reached through a reference: 981 and 4,044 on `skg1` under `schema2` and `schema3`, 4,127 and 17,309 on `mkg1`. (The `maxCount` component total is two higher in each `mkg1` row, from two unrelated `ub:publicationAuthor` failures; the `ub:teacherOf` group itself is 4,127 and 17,309.) The `ub:name` group persists unchanged, at 992 and 968, because nothing in the schema caused it.

The corrected runs also show how much of the referencing workload was itself

an artifact. Failures reached through `sh:node` fall from 8,781 to 655 on `skg1` under `schema2` and from 69,729 to 968 on `mkg1` under `schema3`. The reference chains that the explanation layer exists to reconstruct were therefore, on these schemas, mostly chains into failing courses, and the case for reconstructing them rests on schemas whose referencing structure is not dominated by a single defect.

Interpretation. Because the same failure stories recur through the same reference chains and in the same schema-specific ordering after a fourfold increase in data, they are better understood as properties of the interaction between these schemas and the LUBM data pattern than as artifacts of one graph size. The evaluation includes only two sizes, so this is replication across scale rather than a trend estimate over many scales.

Answer to RQ5. Yes, with two refinements. The same university and course-related failure patterns recur on both datasets through the same reference chains and in the same schema-specific ordering; the university pattern leads only under `schema1`, and under `schema2` and `schema3` the course-related `ub:type` failures are dominant. The second refinement is what those patterns are: the dominant one is an effect of a schema defect and the other of an incompletely described data graph, and the explanation is what made both visible. On the corrected schemas the ranking changes accordingly, with `ub:teacherOf` `maxCount` in first place and the university pattern unchanged behind it.

5.4. Threats to Validity and Summary

This section separates three kinds of limitation. The first concerns what the reported measurements mean, since several of them count something narrower than their names suggest. The second concerns whether the effects reported in Section 5.3 are real, given single runs on a shared machine and schemas supplied rather than written for this evaluation. The third concerns how far the results carry beyond one benchmark family and the constraint components the fixtures reach. A fourth group states what the explanation itself guarantees and where each guarantee stops. The most material limitations are the constraint components no fixture reaches and the absence of deep reference chains at scale; the remainder bound how precisely individual figures should be read rather than what the chapter concludes.

5.4.1. What the reported numbers mean

Uninstrumented work. The instrumented pipeline total excludes interpreter startup, imports, output writing, garbage collection and other work outside the timed stages. This gap is not a constant interpreter-startup cost: it is under 1 second on the smallest

combination and 38 to 43 seconds on the largest, so Table 5.8 should not be read as the exact delay experienced by a user. Writing the output cannot account for it, since the summary run writes nothing to disk and yet shows a gap as large as the HTML run’s. On `mkg1/schema3`, for example, the pipeline total is 642.45 seconds for the HTML run and 631.75 seconds for the summary run, while the two complete invocations take 680 and 675 seconds; Appendix D gives all six combinations in Table D.3. Memory pressure is a plausible contributor because the run holds both a 4.26-million-triple data graph and a report containing 2.28 million validation results, but memory use was not instrumented and this explanation remains conjectural. No conclusion in this chapter rests on the pipeline total as a measure of user-facing latency: the stage shares of RQ4 are computed within the instrumented total and are unaffected by the gap.

What the counts count. Deduplication is applied to the whole explanation forest, and a leaf is identified by its focus node, path, constraint component and value (Section 4.7). Two consequences follow. Two constraints of the same component on the same path in different shapes are treated as one failure. A leaf that is reached from several top-level results is kept once, under the first root that reaches it in the fixed order of Section 4.4, so the output is a view of the distinct failures in a report rather than a per-result view. Table 5.9 reports 189,776 top-level results against 6,306 affected focus nodes for `skg1 × schema2`. That gap is mostly the collapse described above, plus the root-level definition of the metric. It is not a claim that only 6,306 nodes of the data are involved. Adding the source shape to the identity, and recording every root that reaches a leaf, would remove both effects and is left to future work.

Resource limits. The largest HTML report reaches 57 MB, for the combination with the most leaf failures reported in Table 5.9, because the current implementation embeds the complete explanation data in one self-contained document. Appendix D gives all six report sizes in Table D.4. Only time was instrumented, but peak resident memory was measured separately for three runs. It is dominated by the data graph and the report: 11.0 GB on `mkg1` with `schema2` and 5.0 GB on `skg1` with the same schema. Reconstructing from a report without `sh:detail` adds little, since the cache of Section 4.6 holds one small report per distinct pair: 5.2 GB against 5.0 GB on `skg1` with `schema2`, about five percent. Graphs of this size therefore need a machine with memory to spare, and the figures are single measurements on the machine of Section 5.1.3. Browser loading and parsing performance were not measured, so the render timings in Table 5.8 do not capture this user-facing cost. A report may therefore be quick to serialize yet inconvenient to open or distribute. The figures of this chapter therefore assume a machine with this much memory free, and characterize neither behavior under memory pressure nor the cost of opening the resulting report.

5.4.2. Whether the measured effects are real

Measurement noise and its size. The six performance combinations were measured once per output mode rather than repeatedly. No warm-up run was discarded, no variance or confidence interval was computed and the machine was not isolated from other activity. Consequently, small timing differences are not interpreted. Three later passes over all six combinations, run consecutively on mains power with no other load, measure that noise directly. Spread is 0.3 to 3.1 percent across the parse and skg1 stages, but 6.3 percent for validation and 9.2 percent for tree building on mkg1. Every figure in Table 5.8 falls within 4.6 percent of those means, so the single runs are representative; differences below about ten percent on the largest combinations are not. Appendix D gives the three passes per stage in Table D.5. The conclusions rely on far larger effects: validation rises by more than two orders of magnitude across schemas, and the explanation costs below one fifth of it. The tree-cost share reproduces as 9.0 to 19.0 percent against the 9.0 to 19.2 reported here. Individual factors are less stable. The schema1-to-schema3 increase holds at 111 on skg1 but gives 135 to 144 on mkg1, against the 146 above, whose validation was the slowest of four measurements of that combination. Stating differences of that order would need confidence intervals over many runs on an isolated machine.

Schema provenance and the corrections. The three schemas were supplied with the data rather than written for this thesis, and the results above are reported against them as given. The corrected copies are this thesis's own reading of what the audit reports, and two judgements in them are open to disagreement. Removing the `ub:type` property shapes assumes the intent was to constrain the type of a course, which `sh:targetClass` already expresses; an author might instead have meant a datatype property that the data never carries. Pointing `sh:node` at `ub:CourseShape` rather than `ub:Course` assumes the shape was meant, which the existence of a shape with that name makes likely but does not prove. The third correction, `sh:manCount` to `sh:maxCount`, is a misspelling with only one plausible reading, and it changes nothing that this thesis reports: no failure on either graph, and only a duplicate raw result for each node that already exceeds the bound. The requirement of an `ub:name` on universities is deliberately not corrected, since the failures it produces are a property of the generated data. A reader who disagrees with any of these can compare against the supplied-schema results, which are reported in full throughout.

Provenance of the referencing workload. The corrected runs of Section 5.3.5 show that most of the referencing workload measured here came from a schema defect rather than from the schemas' intended structure: failures reached through `sh:node` fall from 8,781 to 655 on skg1 under schema2 and from 69,729 to 968 on mkg1 under schema3 once the `ub:type` shapes are repaired. The performance figures of RQ3 and RQ4 are

therefore reported on reports whose reference chains are dominated by one defect, and the corrected columns of Table 5.10 are the better guide to what the explanation layer costs on a schema that is not so dominated. It also narrows what these runs establish about reconstruction at scale: the chains exercised are many instances of a few shapes rather than a varied referencing structure.

5.4.3. How far the results generalize

Benchmark and topology coverage. Both performance graphs come from one benchmark family and all three schemas use the LUBM vocabulary. The experiment therefore does not establish the same scaling behavior for other domains, data distributions or substantially different shape topologies. In particular, the LUBM schemas admit at most two referencing steps. The large-scale runs vary breadth by nearly three orders of magnitude in result count but do not vary depth. Deep recursion is covered only functionally by TC53’s eight-level chain on a small graph, so the near-linear per-result cost observed in RQ4 should be read as a breadth-scaling result for these schemas rather than as evidence for arbitrary reference depth.

Constraint components not covered. Coverage of constraint components is correspondingly partial. The expander treats every component other than `sh:NodeConstraintComponent` as a leaf, but Table D.2 shows that only `minCount`, `datatype`, `pattern` and `in` were tested as leaves reached through `sh:node`. SHACL-SPARQL constraints, `sh:and` and `sh:qualifiedValueShape` appear in no fixture. `sh:or`, `sh:xone` and `sh:not` are tested only as direct leaves in TC22, and an `sh:node` inside any logical or qualified constraint is reported through the enclosing result rather than traversed (Section 2.3). The complex property-path fixtures likewise occur as direct failures rather than beneath `sh:node`. No result in this chapter should be generalized to those cases.

Correctness at scale. The fixture suite is deliberately small so that expected explanations can be checked exactly. This is a strength for RQ1 and RQ2, but it also limits external validity: the fixtures establish supported patterns mostly in isolation and do not test every possible interaction among them on million-triple graphs. The LUBM runs cannot close that gap because no independent ground-truth explanation tree exists for outputs containing tens of thousands of failures. One check at that scale is nevertheless available and was carried out: with every `sh:detail` triple removed, reconstruction rebuilds the same explanation that reading `sh:detail` produces, on all six combinations and down to the order of the output. That establishes that the two paths agree where one of them is exercised by the fixtures, not that either is right; correctness at scale remains assumed rather than demonstrated in the sense of an independently derived expected explanation.

Report formats. The external reports tested here are Turtle. The command line

accepts RDF/XML, but no real validator's output in that format was tried, so RQ2's result is evidence about Turtle reports from three implementations and not about the serialization-independence of reconstruction.

5.4.4. What the explanation guarantees

Determinism of the output. The explanation is derived from the validation report and the shapes graph by fixed rules, with no generative step, and its output is the same on every run. One kind of variation remains. A complex property path is reported as a blank node that `rdflib` names afresh on every parse, so the path shown for TC30 to TC33 prints a different label on every run while denoting the same path expression. The repair hints of those fixtures describe such a path rather than naming it, precisely because the label would vary between runs and would name nothing to a reader, so they are stable.

The guarantee rests on two rules, and the reason for each is worth stating. Deduplication keeps the first occurrence of a leaf, and the order in which `rdflib` yields results differs from one run to the next even on an identical input file, because Python seeds its string hashing afresh in every process. Without a fixed order, run-to-run chance would therefore decide which root keeps a leaf reached from several roots, and which of its chains is shown as primary; the number of roots, which is what the focus-node count of Table 5.9 measures, would move with it. The results are for that reason put into a fixed order before the tree is built (Section 4.4). The second rule concerns messages: where a shape declares several `sh:message` values, as in TC38 and TC39, all of them are shown in a fixed order, so that the message does not depend on which triple `rdflib` yields first. A test runs TC5, TC26 and TC52 under different hash seeds and TC5 with its triples shuffled, and requires byte-identical output; Appendix B, Section B.5 gives the measurements that established the effect. In every case the set of leaf failures and the chains attached to each are the same. What an unordered walk would change is which of several equivalent routes is shown first, not which failures are reported.

Referenced shape of a step. A reference node records the shape its `sh:node` step points at. If a shape declares several references, the expander identifies the relevant one from the `sh:sourceShape` values of the result's own `sh:detail` children. The validator reports one result per reference, and each result contains only the details produced by the referenced shape for that branch. On the fallback path of Section 4.6, those children are absent. If several references remain possible, the step therefore records no referenced shape rather than guessing. This is the one field reconstruction cannot recover: the failures themselves, their chains, order and messages are rebuilt exactly. A test checks both behaviors, including that reconstruction never attributes a step to a shape contradicted by `sh:detail`.

One ambiguity remains after deduplication. For node-level references, sibling steps are merged when they have the same focus node, declaring shape and preceding reference chain. The combined node retains the referenced-shape value of the first step and adds the children of the other matching steps. This merge is what collapses repeated diamond branches into one subtree. The retained value therefore identifies one reference represented by the combined node, not every route to its children. In TC26, for example, the surviving `ex:WorkerShape` step records `ex:ContractorShape`, but it also contains the `ex:department` failure reached through `ex:StaffShape`. That failure's route remains explicit in its `refChain`, and any alternative routes remain in `altChains`. A reader tracing a particular failure should therefore follow its leaf-level chains rather than infer the route from the enclosing step. Property-level steps with different referenced shapes are not merged in this way because the referenced shape is also their displayed shape and is part of the merge key.

No measurement in this chapter is affected, since no LUBM schema declares more than one `sh:node` on any shape. One presentational inconsistency is left standing: the summary renderer's reference-paths table lists a property-level step as `ex:worksFor -> ex:CompanyShape`, whose arrow points at the referenced shape, but a node-level step as `sh:node -> ex:GradientRootShape`, whose arrow points instead at the shape that declares the reference, as the eight lines of that kind in the committed TC53 summary do. The tree label of Table 4.2 is ambiguous in the same way, but it asserts no direction, whereas the arrow here states one the line itself does not determine.

Path-based recovery. The path fallback of Section 4.6, on which both real validators' property-level reports depend, starts from every property shape that declares `sh:node` on the reported path, and several may. Two limits remain on the narrowing that follows. Targeting is evaluated for the SHACL Core target types, so a shape with a SPARQL-based target is assumed to apply and the narrowing rests on the later steps alone. Some results cannot be told apart by targeting, evidence or declared metadata. If the number of such results also differs from the number of remaining candidates, as happens when a validator truncates its report, no assignment is made and the value is re-validated against every remaining candidate as before. No fixture of Table 5.5 shares a path in this way, so no figure in this chapter is affected.

Narrowing was added because the first version, which used every candidate shape, could fabricate structure: reconstructed from a real TopBraid report, an explanation contained a failure from an unrelated shape and a reference chain that exists nowhere in the schema. Appendix B, Section B.5 gives the probe and its four variants, committed under `external_report_cases/shared_path/` with real reports from both validators. A test asserts each of the eight reconstructions, including that every failure is reached through its own referenced shape alone, and removing any one of the narrowing steps makes it fail.

5.4.5. Reproducibility

The fixtures, automated tests, schemas, commands, generated reports and per-combination breakdown are version controlled. The two LUBM data graphs are not committed because they are 171 MB and 732 MB, so exact reproduction requires the same supplied files; another graph generated from the same benchmark can reproduce the procedure but not necessarily the exact counts. The order of the output, CSV rows included, is fixed; only the labels of blank-node paths differ between runs, as the paragraph on determinism above explains.

5.4.6. Summary of findings

RQ1 shows that all 53 functional fixtures pass under a 52-test suite, at the per-fixture granularity Appendix E records. RQ2 shows that external reports without `sh:detail`, including real reports from Apache Jena and TopBraid, can be reconstructed when the referenced shape or the path leading to it is present, with the expected loss of chain information when it is absent. RQ3 shows that runtime is driven mainly by the number of validation results induced by schema reach rather than by data size alone. RQ4 shows that explanation-tree construction remains a minority of total cost and has an almost constant per-result cost over the measured range. RQ5 shows that the same dominant university and course-related failure patterns recur at both graph sizes, while clarifying that the course clusters overtake the university cluster under the larger schemas. These conclusions are strongest for the explicitly tested SHACL patterns and for breadth scaling on LUBM; repeated performance runs, deeper large-scale schemas, additional domains and browser-level measurements remain necessary for broader claims.

6. Conclusions and Future Work

6.1. Contributions

This thesis localized failures hidden behind SHACL `sh:node` references. Its contributions are:

1. An explanation model for referencing constraints. From a validation report it builds a tree whose interior nodes record reference steps and whose leaves record concrete SHACL Core violations, each leaf retaining the failed component, path, relevant value node and reference chain, with a local repair hint where supported.
2. A reconstruction procedure for reports that omit `sh:detail`, which the reports of Apache Jena and TopBraid do. Targeted re-validation recovers the nested evidence from the referenced shape, or from the reported path where the report identifies only an anonymous property shape.
3. Deduplication that separates a distinct violation from the routes that reach it: a failure reached by several references is kept once, while the alternate chains remain available for inspection.
4. An implementation, evaluated on 53 functional fixtures, on real reports from two independent validators, and on six LUBM dataset–schema combinations.

The tool explains an existing validator’s outcome rather than computing globally minimal repairs, and where targeted re-validation disagrees with an external validator the reported verdict is retained. The output supports diagnosis rather than replacing it: a localized violation gives a concrete place to investigate, but deciding whether to edit data, correct a shape or change a validator configuration still requires domain knowledge.

6.2. What the evaluation shows

Reconstruction does not depend on `sh:detail`. The automated suite covers all 53 fixtures, including property-level and nested references, diamonds, cycles and external

reports, and the diamond cases verify the central distinction between a concrete failure and the routes that reach it: duplicate leaves are collapsed while alternate chains remain available for inspection. On real Apache Jena and TopBraid reports for the referencing fixtures, neither of which carries `sh:detail`, the fallback recovered the same concrete failures as pySHACL’s detailed reports, with one recursive fixture yielding a different chain for one validator. Where a report and shapes graph no longer identify a reference at all, the tool preserves the reported failure but cannot recover its route.

The explanation is a substantial reduction of the report. The largest combination produced 2,281,457 pySHACL validation results but 87,396 distinct leaf failures. This is useful for triage, because intermediate reference results and repeated routes no longer appear as separate concrete problems. It should not be read as a count of independently confirmed data errors: the leaf identity omits the source shape, and deduplication can retain one leaf reached from several top-level results under only the first root. The grouped view answers which distinct violations the current representation exposes, while the original report and the retained chains remain necessary to understand how those violations were reached.

The explanation layer is affordable at the measured scales. Across six LUBM dataset-schema combinations, tree construction took 9.0–19.2% of pySHACL validation time and rendering stayed below 2.8 seconds, while the cost of building the tree per validation result was nearly constant over the measured range.

Cost is governed by schema reach rather than data size. Increasing the data graph roughly fourfold raised validation time four- to fivefold at a fixed schema, whereas broadening the schema on unchanged data raised it by more than two orders of magnitude. The number of nodes a schema’s targets select can be counted before any validation is run, which makes it the practical variable for anticipating cost.

The dominant patterns recur across scale. Missing `ub:name` values reached through degree-related references appeared at both graph sizes, and broader schemas added many missing `ub:type` failures on courses. The latter arose from a defect in the supplied schema and disappeared under the corrected variant; the former reflected a mismatch between university names and descriptions in the generated data and remained. Grouping thousands of results by their reference chains is what exposed both causes. Section 5.4 states the limitations that bound all of these results.

6.3. Future work

6.3.1. Strengthen correctness, then broaden coverage

Section 5.4 sets out three limits of the current deduplication: the leaf identity does not distinguish constraints declared in different source shapes, a shared leaf is retained

under only the first top-level result that reaches it, and the reference-step identity can combine node-level steps that travelled through different referenced shapes. Resolving them is the first task, and it requires something the present suite does not have: independently specified expected explanations, especially for diamonds and shared paths, rather than documents captured from the implementation. Tests should assert the referenced shape, the focus-node transition and every retained route, since checking only leaf counts would miss these errors. Once those invariants hold, the explanation model can be tested against SHACL-SPARQL, logical and qualified constraints and more complex property paths beneath references.

6.3.2. Test reconstruction across validators

Jena and TopBraid already show why synthetic report fixtures are insufficient: their property-level results can identify an anonymous property shape with a fresh blank node and omit `sh:detail`. A wider comparison should use real reports from additional validators, including larger schemas with shared paths and recursion, and measure which failures and reference chains the fallback can reconstruct. Recursive-shape validation is left to processor implementations by SHACL [KK17, Section 3.4.3], so differences in verdicts must be separated from differences in report structure. A preliminary pluggable backend for pySHACL and Jena provides a starting point, while targeted pySHACL re-validation remains necessary when the external report withholds nested results. This work would establish more precisely which report fields a portable explanation requires.

6.3.3. Make large explanations usable

The largest HTML report embeds all explanation data in one document, reaching 57 MB. A preliminary standalone graph-view prototype already consumes the same explanation-tree JSON. Building on it with incremental loading, virtualized lists and graph views, and search over failure groups, could let readers begin working without parsing the entire report first. Evaluation should measure browser load time and memory as well as the time users need to find a concrete failure and choose a repair. Those user-facing measures are needed to determine whether the structural reduction in result count translates into a practical benefit at LUBM scale.

A. Implementation Map

The implementation, its fixture corpus and every generated report discussed in this thesis are available at <https://github.com/mariyakez/Root-Cause-Localization-for-Referencing-Constraints-in-SHACL>. The paths below are relative to that repository's root.

Chapter 4 describes the explanation rules. The corresponding implementation entry points are:

Input and orchestration: `shacl_explainer/cli.py`: `load_graph` and the command-line pipeline.

Report roots: `shacl_explainer/parser.py`: `get_report_node`, `get_top_level_results`.

Tree representation: `shacl_explainer/tree.py`: `ReferenceNode`, `LeafFailure`.

Expansion: `shacl_explainer/expander.py`: `expand_result`, `build_explanation_tree`, `result_path_for`, `referenced_shapes_for`, `display_reference_shape`, `owner_shape_for_property_shape`, `shape_value`, `values_from_graphs`.

Missing detail: `shacl_explainer/fallback.py`: `revalidate_against_shape`, `build_targeted_shapes`, `_copy_shape_recursive`.

Duplicate routes: `shacl_explainer/deduplicator.py`: `deduplicate`, including its local `merge_reference_siblings` helper.

Output: `shacl_explainer/renderer.py`: `full_leaf_chain`, `configure_prefixes`, `short_uri`, `tree_to_data`; `shacl_explainer/report_template.html`: `enrichGraphWithCatalog`.

B. Worked Examples and Reconstruction Cases

The following examples expand the rules in Chapter 4 without interrupting the main pipeline description.

B.1. Reference display rules and text examples

Table 4.2 gives the property-level and node-level display rules. The examples below show their effect on the rendered text.

The renderer normally prints one shape label per reference step: the declaring shape for a node-level step and the referenced shape plus path for a property-level step. To make both ends visible, the text examples below use a fuller arrow notation. Only those labels and ASCII tree characters differ from the renderer output. The renderer also records the referenced shape separately. When several candidates exist, the expander identifies it from the `sh:sourceShape` values of the result's own `sh:detail` children. If the report does not contain enough detail to distinguish the candidates, the field is omitted. Deduplication can later combine node-level steps that travelled through different references. The combined step retains one referenced-shape value, while each leaf's `refChain` records its own route; Section 5.4 discusses this ambiguity.

The first text example starts with TC7, which records `path=ex:worksFor` and attributes the concrete `ex:legalName` failure to `ex:acme`. TC3 is included only to show the node-level contrast: `ex:carol` remains the focus node through both reference steps.

```
// TC7: property-level reference
Focus node: ex:alice/
+-- ex:EmployeeShape --sh:node[path=ex:worksFor]--> ex:CompanyShape/
    +-- [failure] minCount ex:legalName = (missing)
        |-- via ex:CompanyShape
        |-- message: ex:legalName is required
    +-- -> fix: Add at least one ex:legalName value to ex:acme.
```

```
// TC3: node-level contrast
Focus node: ex:carol/
+-- ex:ContractorShape --sh:node--> ex:EmployeeShape/
  +-- ex:EmployeeShape --sh:node--> ex:PersonShape/
    +-- [failure] datatype ex:age = "thirty-one"
      |-- via ex:ContractorShape -> ex:EmployeeShape -> ex:PersonShape
      |-- message: ex:age must be an xsd:integer
      +-- -> fix: Replace "thirty-one" on ex:age with a value of type xsd:
            integer.
```

Text examples: TC7 followed by a node-level contrast from TC3. Each reference step uses the two-shape notation introduced above; Table 4.2 gives the single-shape display rule used by the renderer. In TC7 the concrete failure belongs to the property value, not the outer focus node.

The two reference kinds also need different display labels. A property-level step shows the referenced shape with its path; a node-level chain keeps the successive outer shapes visible, rather than collapsing the route to its final target. Path, referenced shape and display shape are therefore resolved separately at each step.

B.2. TC7 expansion trace

Worked trace (TC7). Applying Procedure 1 to the top-level result for the TC7 text example above, where `ex:alice` `ex:worksFor` `ex:acme` and `ex:acme` is missing `ex:legalName`:

1. The result gives focus node `ex:alice`, source constraint component `sh:NodeConstraintComponent`, source shape `ex:EmployeeShape` and path `ex:worksFor`.
2. The component is `sh:NodeConstraintComponent`, so this is not a leaf; expansion continues.
3. A path is present, so this is a property-level step: the displayed shape is the referenced shape `ex:CompanyShape`, with path `ex:worksFor`.
4. The pair (`ex:alice`, `ex:CompanyShape`) has not been visited, so it is added to a fresh visited set for the recursive call.
5. A `ReferenceNode` is created with focus node `ex:alice`, displayed shape `ex:CompanyShape` and path `ex:worksFor`.
6. `pySHACL` attached one nested `sh:detail` result, the `minCount` violation on `ex:legalName`, so it is expanded directly with the chain extended to (`ex:CompanyShape`).

7. Skipped: Procedure 2 is only invoked when `sh:detail` is absent, which is not the case here.
8. The populated `ReferenceNode` is returned with one child. That child re-enters Procedure 1: its component is `sh:MinCountConstraintComponent`, so the component check turns it directly into the `LeafFailure` shown in the TC7 text example, with focus node `ex:acme` and reference chain (`ex:CompanyShape`).

B.3. Incomplete reports: what survives

What a report withholds varies, and so does how much of the reference survives it. Table B.1 sets out the four cases the fallback distinguishes, ordered by how much the report gives up, together with the fixtures that exercise each. Section 4.6 gives the recovery rules, and Section 5.3.2 evaluates these cases.

Table B.1.: What an incomplete external report withholds, and what the fallback still recovers. The rows are ordered by how much the report gives up. Only the first names a shape that resolves in the shapes graph; the last names none at all.

What the report withholds	What the reconstruction still recovers	Fixtures
<code>sh:detail</code> alone. The shape named in <code>sh:sourceShape</code> resolves in the shapes graph, and the result is node-level, so no path is needed.	Everything. The referenced shape is read from that shape's own <code>sh:</code> node; where it declares none, the named shape is itself what re-validation applies. The target is the focus node, and re-validation supplies the leaves the report omitted.	<code>external_report_tc2_no_detail.ttl</code> , TC42, and both validators' node-level reports
<code>sh:detail</code> , and the shape that declares the reference. <code>sh:sourceShape</code> names the <i>enclosing</i> node shape, with <code>sh:resultPath</code> identifying the failing property.	Everything. <code>property_shapes_for</code> reads the referenced shape from the property shape that the named node shape owns on the reported path. The target is the reported <code>sh:</code> value, or the values found along the path when it is absent.	<code>synthetic_report_tc7_enclosing_shape_no_detail.ttl</code> , TC43
<code>sh:detail</code> , and any usable <code>sh:sourceShape</code> . The shape reported identifies nothing that declares the reference: both real validators name the property shape, anonymous in the usual idiom and therefore a fresh blank node, while TC44 names a shape that exists but declares no reference on the path.	Everything, through the path alone. <code>property_shapes_for_unresolved</code> collects every property shape declaring <code>sh:node</code> on the reported path and narrows them by the three steps below. This is the case the real reports actually take, and the one the hand-written fixtures alone would leave untested.	<code>jena_real_tc7_property_no_detail.ttl</code> , <code>topbraid_real_tc7_property_no_detail.ttl</code> , TC44, <code>shared_path/</code>
<code>sh:detail</code> and <code>sh:sourceShape</code> both.	The leaf failure only. Nothing in the report identifies a reference, so no reference step is shown; the failing path, the value and the repair hint are still read from the result itself.	TC45

The last row of the table is the boundary case, and TC45 works it through. The algorithm degrades gracefully when information is insufficient. A direct Core result that

omits `sh:sourceShape` can still become a leaf because its focus node, path, component and message remain meaningful. The following TC45 output exercises this boundary:

```
Focus node: ex:alice/  
+-- [failure] minCount ex:name = (missing)  
    +-- message: External report omitted sh:sourceShape
```

TC45 text example: Graceful degradation for TC45. The concrete failure survives, but no reference chain is invented when `sh:sourceShape` is unavailable.

B.4. TC7 JSON example

Structured output. JSON and HTML consume one serialized tree of dictionaries. The following abbreviated TC7 example shows the outer reference and its concrete leaf.

```
[  
  {  
    "type": "reference",  
    "focusNode": "ex:alice",  
    "shape": "ex:CompanyShape",  
    "referencedShape": "ex:CompanyShape",  
    "path": "ex:worksFor",  
    "children": [  
      {  
        "type": "leaf",  
        "focusNode": "ex:acme",  
        "path": "ex:legalName",  
        "component": "sh:MinCountConstraintComponent",  
        "message": "ex:legalName is required",  
        "repairHint": "Add at least one ex:legalName value to ex:acme.",  
        "refChain": ["ex:CompanyShape"],  
        "altChains": []  
      }  
    ]  
  }  
]
```

JSON example: Abbreviated output for TC7. The outer reference remains about `ex:alice`, while the leaf records `ex:acme` as the node on which the concrete constraint failed.

B.5. Evaluation probes: ordering and shared paths

Two measurements referred to in Section 5.4 are recorded here rather than in the chapter, because each is a diagnostic run rather than a reported result.

Ordering. Before the results were put into a fixed order, the output varied between runs on an identical input file. Three runs on TC52 produced its ten focus-node roots in three different orders. On TC5 the primary route switched between `ex:FulltimeShape` and `ex:ParttimeShape` with the order of the triples in the file, and the number of roots moved with it, which is the quantity the focus-node count of Table 5.9 measures. The set of leaf failures and the chains attached to each were the same in every run; what varied was which of several equivalent routes was shown first.

Shared paths. The path fallback starts from every property shape that declares `sh:node` on the reported path, and its first version used all of them. The probe that showed the consequence declares an `ex:EmployeeShape` and an unrelated `ex:VolunteerShape`, both constraining `ex:worksFor`, to `ex:CompanyShape` and `ex:CharityShape` respectively. Reconstructed from TopBraid’s report, the explanation of an employee’s employer contained the correct `legalName` failure and a spurious `charity Number` failure. Where the focus node was both an employee and a volunteer, it also joined the two referenced shapes into a chain `ex:CompanyShape -> ex:CharityShape` that exists nowhere in the schema. Four variants are committed under `external_report_cases/shared_path/` with real reports from both validators, and all four reproduce pySHACL’s native explanation once the candidates are narrowed: only the employee’s shape applying, both applying and both failing, both applying with declared messages, and both applying with only one failing.

C. CSV Export Columns

Section 4.8 describes the CSV output mode, which flattens the explanation tree to one row per distinct leaf failure. Table C.1 lists the ten columns written by `write_csv`.

Table C.1.: The ten columns written by `write_csv`, one row per distinct leaf failure.

Column	Content
<code>focus_node</code>	The node on which the concrete constraint was evaluated
<code>depth</code>	Number of <code>sh:node</code> steps traversed to the leaf (Section 4.8)
<code>reference_chain</code>	The primary route of referenced shapes leading to the leaf
<code>alternate_reference_chains</code>	Other routes to the same leaf, joined by a vertical bar
<code>leaf_path</code>	The failing <code>sh:path</code>
<code>component</code>	The violated SHACL Core constraint component
<code>value</code>	The offending value, where available
<code>message</code>	The validator's result message
<code>kind</code>	referenced when the chain is non-empty, direct otherwise
<code>repair_hint</code>	The generated local repair hint, where applicable

Example rows. The TC5 diamond produces four rows, the first of which is shown. `ex:eve` fails the same referenced shape by two routes, so the leaf is written once and the second route is carried in `alternate_reference_chains` rather than as a duplicate row; that column is empty on the three rows reached by one route only. Each record is one line in the file, wrapped here for the page.

```
focus_node,depth,reference_chain,alternate_reference_chains,leaf_path,
component,value,message,kind,repair_hint
ex:eve,2,ex:StaffShape -> ex:FulltimeShape -> ex:PersonShape,
ex:StaffShape -> ex:ParttimeShape -> ex:PersonShape,ex:name,
sh:MinCountConstraintComponent,,ex:name is required,referenced,
Add at least one ex:name value to ex:eve.
```

D. Supplementary Evaluation Tables

These tables provide the fixture coverage and supporting measurements for Chapter 5. Per-fixture assertions are listed in Appendix E.

Table D.1.: The ten categories of numbered fixtures in `all_test_cases/`, covering 53 test cases in total. The directory holds one further category, `repair_hint_cases/`, whose fixtures are named rather than numbered and cover repair hints the numbered corpus does not reach.

Category	Cases	Purpose
<code>sh_node_cases</code>	TC1 to TC8, TC53	Nested, branched, property-level, cyclic, diamond and eight-level references
<code>direct_constraint_cases</code>	TC9 to TC23	Direct SHACL Core failures
<code>mixed_constraint_cases</code>	TC24 to TC26	Direct and referenced failures
<code>multi_focus_cases</code>	TC27 to TC29	Multiple focus nodes, repeated violations and shapes
<code>property_path_cases</code>	TC30 to TC33	Inverse, sequence, alternative and zero-or-more property paths
<code>severity_cases</code>	TC34 to TC36	Violation, Warning and Info severity
<code>message_metadata_cases</code>	TC37 to TC40	Missing, multiple and tagged messages; blank shapes
<code>external_report_cases</code>	TC41 to TC45	Detailed and incomplete external reports
<code>cycle_cases</code>	TC46 to TC49	Self and mutual recursion
<code>scale_cases</code>	TC50 to TC52	Many focus nodes and values; repeated diamond

D. Supplementary Evaluation Tables

Table D.2.: Fixture component coverage: direct failures and leaves beneath `sh:node`. A dash means no case.

Label	As a direct failure	As a leaf beneath <code>sh:node</code>
<code>minCount</code>	TC9	TC1 to TC8, TC53 and others
<code>maxCount</code>	TC10, TC51	—
<code>datatype</code>	TC11, TC23	TC1 to TC6, TC53 and others
<code>class</code>	TC12, TC33	—
<code>pattern</code>	TC13, TC23	TC2, TC6, TC53
<code>in</code>	TC14, TC23	TC6, TC53
<code>nodeKind</code>	TC15	—
<code>minInclusive</code>	TC16	—
<code>maxInclusive</code>	TC16	—
<code>lessThan</code>	TC17	—
<code>hasValue</code>	TC18	—
<code>equals</code>	TC19	—
<code>disjoint</code>	TC20	—
<code>closed</code>	TC21	—
<code>or</code>	TC22	—
<code>xone</code>	TC22	—
<code>not</code>	TC22	—

Table D.3.: Instrumented pipeline and complete wall-clock times (seconds).

Dataset $\times$ Schema	HTML run total	HTML run wall	Summary run total	Summary run wall
<code>skg1</code> $\times$ <code>schema1</code>	20.16	21	20.03	21
<code>skg1</code> $\times$ <code>schema2</code>	91.84	96	91.00	95
<code>skg1</code> $\times$ <code>schema3</code>	129.84	135	129.20	134
<code>mkg1</code> $\times$ <code>schema1</code>	87.78	94	86.32	91
<code>mkg1</code> $\times$ <code>schema2</code>	438.19	464	416.26	443
<code>mkg1</code> $\times$ <code>schema3</code>	642.45	680	631.75	675

Table D.4.: HTML report sizes by leaf-failure count (rounded).

Dataset $\times$ Schema	Leaf failures	HTML report size
<code>skg1</code> $\times$ <code>schema1</code>	992	664K
<code>skg1</code> $\times$ <code>schema2</code>	10,099	6.1M
<code>skg1</code> $\times$ <code>schema3</code>	21,170	14M
<code>mkg1</code> $\times$ <code>schema1</code>	968	756K
<code>mkg1</code> $\times$ <code>schema2</code>	39,736	24M
<code>mkg1</code> $\times$ <code>schema3</code>	87,396	57M

D. Supplementary Evaluation Tables

Table D.5.: Run-to-run spread over three repeat passes, in seconds, run consecutively on mains power with no other load. *Spread* is the range divided by the mean. The passes were run in summary mode, so the render stage is omitted: its cost differs by output mode and is not comparable with the render column of Table 5.8.

Dataset $\times$ Schema	Stage	Pass 1	Pass 2	Pass 3	Mean	Spread
skg1 $\times$ schema1	Parse	18.90	19.31	19.10	19.10	2.1%
skg1 $\times$ schema1	Validate	0.827	0.817	0.822	0.822	1.2%
skg1 $\times$ schema1	Build tree	0.093	0.095	0.096	0.095	3.1%
skg1 $\times$ schema2	Parse	18.88	18.85	18.75	18.83	0.7%
skg1 $\times$ schema2	Validate	61.36	61.41	61.09	61.29	0.5%
skg1 $\times$ schema2	Build tree	10.18	10.29	10.04	10.17	2.4%
skg1 $\times$ schema3	Parse	19.09	18.80	18.85	18.92	1.5%
skg1 $\times$ schema3	Validate	92.44	91.92	91.25	91.87	1.3%
skg1 $\times$ schema3	Build tree	17.60	17.41	17.40	17.47	1.2%
mkg1 $\times$ schema1	Parse	83.27	83.19	83.44	83.30	0.3%
mkg1 $\times$ schema1	Validate	3.26	3.28	3.28	3.27	0.6%
mkg1 $\times$ schema1	Build tree	0.295	0.297	0.295	0.295	0.7%
mkg1 $\times$ schema2	Parse	84.89	82.70	83.72	83.77	2.6%
mkg1 $\times$ schema2	Validate	303.47	287.20	285.03	291.90	6.3%
mkg1 $\times$ schema2	Build tree	50.42	46.94	46.01	47.79	9.2%
mkg1 $\times$ schema3	Parse	83.83	83.44	83.16	83.47	0.8%
mkg1 $\times$ schema3	Validate	469.82	451.30	441.40	454.17	6.3%
mkg1 $\times$ schema3	Build tree	83.88	84.25	82.23	83.45	2.4%

E. Per-Fixture Assertion Coverage

Section 5.3.1 reports what the automated suite establishes for RQ1. Table E.1 gives that evidence per fixture, so that a claim about one fixture can be checked against what is actually asserted for it rather than against the suite as a whole.

The second column distinguishes two kinds of oracle. *Fields* means that the test names specific strings that must appear in the command’s output, listed in the third column; anything the test does not name is unchecked for that fixture. *Stored JSON* means that the fixture’s complete JSON explanation is compared against a document committed under `tests/expected_json/`, so every field of every node participates in the comparison. The stored documents were introduced because the twenty-five fixtures now marked *stored JSON* previously had no content assertion at all: the only test that reached them asserted that the command produced a summary or reported the data as valid.

The comparison against a stored document is order-insensitive. Root order and sibling order vary between runs, for the reasons given in Section 5.4, so both sides are sorted by content before they are compared. One further normalization is recorded in the table itself. A complex property path is reported as the label of a blank node, which `rdflib` mints afresh on every parse, so those labels are replaced by a placeholder; the comparison therefore establishes that the path is a blank-node expression, not which expression it is. TC38 and TC39, whose shapes declare two `sh:message` values each, are compared like the rest: the explanation shows both messages, sorted, and a separate test checks that repeated runs agree.

Table E.1.: Asserted content per fixture. *Fields* means the named strings must appear in the output; *stored JSON* means the whole explanation document is compared.

Case	Oracle	Asserted content
TC1	fields	step label, component, path, value
TC2	fields	step label, 3 components, 3 paths, value, leaf-failure count, repair hints, CSV header
TC3	fields	both step labels, component, path, value
TC4	fields	step label, 3 components, 3 paths, value
TC5	fields	step label, both branch shapes, 4 components, 4 paths, alternate chain, one-copy counts
TC6	fields	3 step labels, focus node, 6 components, 6 paths, 3 full chains, one-copy counts, CSV chain, HTML content
TC7	fields	step label with path, focus node, component, path, message, repair hint, value-node target; the same fields again from the real Jena and TopBraid reports

E. Per-Fixture Assertion Coverage

Case	Oracle	Asserted content
TC8	fields	leaf-failure count line, path
TC9	fields	component, path, zero <code>sh:node</code> leaves
TC10	fields	component, path, zero <code>sh:node</code> leaves
TC11	fields	component, path, zero <code>sh:node</code> leaves
TC12	fields	component, path, zero <code>sh:node</code> leaves
TC13	fields	component, path, zero <code>sh:node</code> leaves
TC14	fields	component, path, zero <code>sh:node</code> leaves
TC15	fields	component, path, zero <code>sh:node</code> leaves
TC16	fields	2 components, path, zero <code>sh:node</code> leaves
TC17	fields	component, path, zero <code>sh:node</code> leaves
TC18	fields	component, path, zero <code>sh:node</code> leaves
TC19	fields	component, path, zero <code>sh:node</code> leaves
TC20	fields	component, path, zero <code>sh:node</code> leaves
TC21	fields	component, path, zero <code>sh:node</code> leaves
TC22	fields	3 components, zero <code>sh:node</code> leaves
TC23	fields	4 components, zero <code>sh:node</code> leaves
TC24	stored JSON	every leaf: focus node, path, component, value, message, repair hint, chains
TC25	stored JSON	as above
TC26	stored JSON	as above
TC27	stored JSON	as above
TC28	stored JSON	as above
TC29	stored JSON	as above
TC30	stored JSON	as above; path is a blank-node expression and is compared as such
TC31	stored JSON	as above; path is a blank-node expression and is compared as such
TC32	stored JSON	as above; path is a blank-node expression and is compared as such
TC33	stored JSON	as above; path is a blank-node expression and is compared as such
TC34	stored JSON	as above
TC35	stored JSON	as above
TC36	stored JSON	as above
TC37	stored JSON	as above
TC38	stored JSON	as above; the message holds both declared values, and repeated runs must agree
TC39	stored JSON	as above; the message holds both declared values, and repeated runs must agree
TC40	stored JSON	as above
TC41	fields	summary produced, path
TC42	fields	summary produced, path
TC43	fields	summary produced, path
TC44	fields	summary produced, path
TC45	fields	summary produced, path
TC46	stored JSON	every leaf: focus node, path, component, value, message, repair hint, chains
TC47	stored JSON	as above
TC48	stored JSON	as above
TC49	stored JSON	empty explanation forest, i.e. the data is reported as valid; the real TopBraid report must also be reported as valid
TC50	stored JSON	every leaf: focus node, path, component, value, message, repair hint, chains
TC51	stored JSON	as above
TC52	stored JSON	as above, for all twenty deduplicated leaves
TC53	stored JSON	as above, for all sixteen leaves across eight levels

F. Corrections to the Supplied LUBM Schemas

Section 5.2.1 reports the defects that `tools/audit_shacl_schema.py` finds in the supplied schemas, and Section 5.3.3 uses corrected copies to separate their effect from the effect of schema reach. This appendix records exactly what the corrected copies change, so that the comparison can be checked or disputed. The files are `lubm_schemas/corrected/schema2.ttl` and `lubm_schemas/corrected/schema3.ttl`; `schema1` produced no findings and is used unchanged.

1. **The `ub:type` property shapes are removed**, from `ub:CourseShape` in `schema2` and from `ub:CourseShape` and `ub:GraduateCourseShape` in `schema3`. Each declared `sh:path ub:type` with `sh:minCount 1`, and LUBM has no `ub:type` predicate. The `sh:value` triple beside each of them is removed with the block; `sh:value` is a term of the validation-result vocabulary and no constraint, so it never had an effect.
2. **Class-valued `sh:node` references are pointed at the shapes of those classes**: `sh:node ub:Course` becomes `sh:node ub:CourseShape` and `sh:node ub:GraduateCourse` becomes `sh:node ub:GraduateCourseShape`. One reference is affected in `schema2` and eight in `schema3`.
3. `sh:manCount` **becomes** `sh:maxCount` on `ub:FullProfessorShape` and `ub:LecturerShape` in `schema3`, a misspelling that validators ignore. The upper bound is not lost: each shape declares a second property shape on `ub:teacherOf` that spells `sh:maxCount` correctly, and `sh:maxCount` counts every value of the path. Correcting the misspelling therefore duplicates the existing result rather than adding a restriction.

The corrections touch five triples in `schema2` and eighteen in `schema3`: four removed and one rewritten in the first, eight removed and ten rewritten in the second, so each file ends up four and eight triples shorter. The audit reports no finding on either corrected file. Nothing else is altered: in particular `ub:UniversityShape` still requires an `ub:name`, for the reason given in Section 5.2.1.

List of Figures

2.1. Property-level sh:node in TC7.	9
2.2. Labelled explanation tree introducing the terminology of this thesis. . .	10
4.1. Architecture of the explainer: the five stages of the pipeline.	18
4.2. TC7 shapes graph before and after targeted reconstruction.	27
4.3. Targeted re-validation when an external report omits sh:detail: one validation per shape–target pair.	31
4.4. Deduplication of the TC5 diamond, before and after. Two same-labelled root steps merge, two duplicate leaves are folded into alternate chains, and the resulting childless ex:ParttimeShape node is pruned.	35
4.5. Sidebar navigation in three modes on TC52.	40
4.6. Property-level sh:node in the analytical view, combining the reference- step and leaf-failure node blocks and showing that the concrete failure is attributed to the value node reached through the reference path.	41
4.7. Graph view of a nested reference chain, with reference depth encoded visually along the chain.	41
4.8. The same graph in Violations only and All constraints modes. The latter supplements the explanation with non-violating constraints from the shapes graph.	42

List of Tables

2.1. Validation-result properties used in this thesis, adapted from	6
2.2. SHACL Core constraint components discussed in this thesis.	7
3.1. Comparison of this thesis with closely related tools.	16
4.1. The two node types used by the explanation tree.	19
4.2. Property-level versus node-level display rules.	24
4.3. The five presentations consume the same explanation tree.	36
4.4. Constraint components for which the renderer produces a repair hint. Other components remain fully reported but receive no generic hint. . .	38
5.1. The two LUBM-derived data graphs used in the performance evaluation. Throughout this chapter they are abbreviated skg1 and mkg1.	46
5.2. The three SHACL shape schemas, in increasing order of size. Each data graph in Table 5.1 is validated against each of them, giving the six combinations reported in Section 5.3.	47
5.3. Schema reach: the distinct focus nodes each schema selects in each data graph, counted before validation.	48
5.4. Defects reported by the schema audit, with the effect each has on validation.	48
5.5. Outcomes on the referencing fixtures. Depth is the number of sh:node steps traversed to the deepest leaf, as printed by the summary renderer. TC49 conforms, so no explanation tree is produced.	51
5.6. Reconstruction from external validation reports. TC41 is the control in which sh:detail is present.	53
5.7. Cost of reconstruction when sh:detail is removed, against the normal path that reads it. Calls counts the pySHACL re-validations.	57
5.8. Runtime breakdown per stage, in seconds, one run per combination. . .	58
5.9. pySHACL result counts and explanation output counts per combination.	59
5.10. Supplied against corrected schemas, one summary run each.	60
5.11. Cost of the explanation layer against two baselines. Ratios are derived from the measurements in Table 5.8.	62
5.12. Largest leaf-failure groups by reference chain. Entries marked n/a correspond to shapes not present in that schema.	64

List of Tables

B.1. What an incomplete external report withholds, and what the fallback still recovers.	80
C.1. The ten columns written by write_csv, one row per distinct leaf failure.	83
D.1. The ten categories of numbered fixtures in all_test_cases/, covering 53 test cases in total.	84
D.2. Fixture component coverage.	85
D.3. Pipeline and wall-clock times.	85
D.4. HTML report sizes.	85
D.5. Run-to-run spread over three repeat passes.	86
E.1. Asserted content per fixture. Fields means the named strings must appear in the output; stored JSON means the whole explanation document is compared.	87

List of Procedures

1.	Expanding a validation result	22
2.	Reconstructing a <code>sh:node</code> result without <code>sh:detail</code>	30

Bibliography

- [Ahm+21] S. Ahmetaj, R. David, M. Ortiz, A. Polleres, B. Shehu, and M. Šimkus. “Reasoning about Explanations for Non-validation in SHACL.” In: *Proceedings of the 18th International Conference on Principles of Knowledge Representation and Reasoning*. 2021, pp. 12–21. doi: 10.24963/kr.2021/2.
- [Ahm+22] S. Ahmetaj, R. David, A. Polleres, and M. Šimkus. “Repairing SHACL Constraint Violations Using Answer Set Programming.” In: *The Semantic Web – ISWC 2022*. Vol. 13489. Lecture Notes in Computer Science. Cham: Springer, 2022, pp. 375–391. doi: 10.1007/978-3-031-19433-7_22.
- [Ahm+25] S. Ahmetaj, R. David, A. Polleres, and M. Šimkus. “A Logic Programming Approach to Repairing SHACL Constraint Violations.” In: *Transactions on Graph Data and Knowledge* 3.3 (2025), 1:1–1:36. doi: 10.4230/TGDK.3.3.1.
- [Ahm+26] S. Ahmetaj, I. Boneva, J. Hidders, M. Jakubowski, J. E. Labra Gayo, W. Martens, F. Mogavero, F. Murlak, C. Okulmus, O. Savković, M. Šimkus, and D. Tomaszuk. “Common Foundations for Recursive Shape Languages.” In: *Proceedings of the 23rd International Conference on Principles of Knowledge Representation and Reasoning*. 2026, pp. 2–13. doi: 10.24963/kr.2026/1.
- [AMP24] S. Ahmetaj, T. C. Merkl, and R. Pichler. “Consistent Query Answering over SHACL Constraints.” In: *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning*. 2024, pp. 2–13. doi: 10.24963/kr.2024/1.
- [Apa26] Apache Software Foundation. *Apache Jena*. Version 6.2.0. July 31, 2026.
- [BJ21] B. Bogaerts and M. Jakubowski. “Fixpoint Semantics for Recursive SHACL.” In: *Proceedings of the 37th International Conference on Logic Programming (Technical Communications)*. Vol. 345. Electronic Proceedings in Theoretical Computer Science. Open Publishing Association, 2021, pp. 41–47. doi: 10.4204/EPTCS.345.14.
- [BJV22] B. Bogaerts, M. Jakubowski, and J. Van den Bussche. “SHACL: A Description Logic in Disguise.” In: *Logic Programming and Nonmonotonic Reasoning*. Vol. 13416. Lecture Notes in Computer Science. Cham: Springer, 2022, pp. 75–88. doi: 10.1007/978-3-031-15707-3_7.

- [CRS18] J. Corman, J. L. Reutter, and O. Savković. “Semantics and Validation of Recursive SHACL.” In: *The Semantic Web – ISWC 2018*. Vol. 11136. Lecture Notes in Computer Science. Cham: Springer, 2018, pp. 318–336. doi: 10.1007/978-3-030-00671-6_19.
- [CWL14] R. Cyganiak, D. Wood, and M. Lanthaler. *RDF 1.1 Concepts and Abstract Syntax*. W3C. 2014. URL: <https://www.w3.org/TR/rdf11-concepts/>.
- [De +21] B. De Meester, P. Heyvaert, D. Arndt, A. Dimou, and R. Verborgh. “RDF Graph Validation Using Rule-Based Reasoning.” In: *Semantic Web 12.1* (2021), pp. 117–142. doi: 10.3233/SW-200384.
- [Del+23] T. Delva, A. Dimou, M. Jakubowski, and J. Van den Bussche. “Data Provenance for SHACL.” In: *Proceedings of the 26th International Conference on Extending Database Technology*. OpenProceedings.org, 2023, pp. 285–297. doi: 10.48786/edbt.2023.23.
- [FRV21] M. Figuera, P. D. Rohde, and M.-E. Vidal. “Trav-SHACL: Efficiently Validating Networks of SHACL Constraints.” In: *Proceedings of the Web Conference 2021*. ACM / IW3C2, 2021, pp. 3337–3348. doi: 10.1145/3442381.3449877.
- [GPH05] Y. Guo, Z. Pan, and J. Heflin. “LUBM: A Benchmark for OWL Knowledge Base Systems.” In: *Journal of Web Semantics* 3.2–3 (2005), pp. 158–182. doi: 10.1016/j.websem.2005.06.005.
- [KK17] H. Knublauch and D. Kontokostas. *Shapes Constraint Language (SHACL)*. W3C. 2017. URL: <https://www.w3.org/TR/shacl/>.
- [Lab+18] J. E. Labra Gayo, E. Prud’hommeaux, I. Boneva, and D. Kontokostas. *Validating RDF Data*. Vol. 7. Synthesis Lectures on the Semantic Web: Theory and Technology 1. Morgan & Claypool, 2018, pp. 1–328. doi: 10.2200/S00786ED1V01Y201707WBE016.
- [Mäk+25] J. Mäkelburg, Z. Zacouris, J. Ke, and M. Acosta. “Interactive Analysis of Knowledge Graph Validation Results with the SHACL Dashboard.” In: *ISWC 2025 Companion Volume*. Vol. 4085. CEUR Workshop Proceedings. 2025, pp. 448–454.
- [Mäk+26] J. Mäkelburg, Z. Zacouris, J. Ke, and M. Acosta. “SHACL Dashboard: Analyzing Data Quality Reports Over Large-Scale Knowledge Graphs.” In: *The Semantic Web – ISWC 2025*. Vol. 16141. Lecture Notes in Computer Science. Cham: Springer, 2026, pp. 96–112. doi: 10.1007/978-3-032-09530-5_6.
- [PC14] E. Prud’hommeaux and G. Carothers. *RDF 1.1 Turtle: Terse RDF Triple Language*. W3C. 2014. URL: <https://www.w3.org/TR/turtle/>.

- [PK22] P. Pareti and G. Konstantinidis. “A Review of SHACL: From Data Validation to Schema Reasoning for RDF Graphs.” In: *Reasoning Web. Declarative Artificial Intelligence*. Vol. 13100. Lecture Notes in Computer Science. Cham: Springer, 2022, pp. 115–144. doi: 10.1007/978-3-030-95481-9_6.
- [PKM22] P. Pareti, G. Konstantinidis, and F. Mogavero. “Satisfiability and Containment of Recursive SHACL.” In: *Journal of Web Semantics* 74 (2022), p. 100721. doi: 10.1016/j.websem.2022.100721.
- [PL25] G. C. Publio and J. E. Labra Gayo. “xpSHACL: Explainable SHACL Validation Using Retrieval-Augmented Generation and Large Language Models.” In: *Proceedings of the 2nd LLM+Graph Workshop, co-located with VLDB 2025*. 2025.
- [PLS14] E. Prud’hommeaux, J. E. Labra Gayo, and H. Solbrig. “Shape Expressions: An RDF Validation and Transformation Language.” In: *Proceedings of the 10th International Conference on Semantic Systems*. ACM, 2014, pp. 32–40. doi: 10.1145/2660517.2660523.
- [RLH22] K. Rabbani, M. Lissandrini, and K. Hose. “SHACL and ShEx in the Wild: A Community Survey on Validating Shapes Generation and Adoption.” In: *Companion Proceedings of the Web Conference 2022*. ACM, 2022, pp. 260–263. doi: 10.1145/3487553.3524253.
- [SC26] A. Sommer and N. Car. *pySHACL*. Version 0.31.0. 2026. doi: 10.5281/zenodo.4750840.
- [Ste+26] C. A. Steinparz, A. Hinterreiter, L. Bajraktari, V. Sedlyarov, M. J. Bauer, T. Zichner, and M. Streit. “SHACLens: A Visualization Workflow for SHACL Violation Exploration in Knowledge Graphs.” In: *Frontiers in Bioinformatics* 6 (2026), p. 1756507. doi: 10.3389/fbinf.2026.1756507.
- [Top26] TopQuadrant. *TopBraid SHACL API*. Version 1.5.0. Mar. 23, 2026.